

# Mitigating Undercutting Attacks: A Study on Mining and Transaction Fee Behavior

by

Claire Bao

S.B. Computer Science and Engineering, Massachusetts Institute of Technology, 2024

Submitted to the Department of Electrical Engineering and Computer Science  
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER  
SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2024

© 2024 Claire Bao. This work is licensed under a [CC BY-NC-ND 4.0](#) license.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Claire Bao  
Department of Electrical Engineering and Computer Science  
May 10, 2024

Certified by: Neha Narula  
Director, Digital Currency Initiative, Thesis Supervisor

Accepted by: Katrina LaCurts  
Chair  
Master of Engineering Thesis Committee



# Mitigating Undercutting Attacks: A Study on Mining and Transaction Fee Behavior

by

Claire Bao

Submitted to the Department of Electrical Engineering and Computer Science  
on May 10, 2024 in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND COMPUTER  
SCIENCE

## ABSTRACT

With block rewards dwindling in Bitcoin, a miner's revenue will become increasingly reliant on transaction fees. However, these transaction fees are highly variable, which could result in undercutting attacks occurring. Undercutting attacks are when miners intentionally fork the blockchain in an attempt to steal transactions from an already-mined block. These attacks could cause repeated forking of the blockchain, thereby rendering Bitcoin unstable and less secure long-term. The original paper [1] proposing these attacks made assumptions about the future mining environment. For instance, they assumed that block size limits were large relative to the number of transactions and that all transactions had the same fee.

This thesis aims to examine whether undercutting attacks would still be a threat under different mining dynamics. Specifically, we examine two important mempool characteristics that have changed since the original paper was written: the block size limit and the fee gradient. By investigating what happens as these characteristics and factors change, our research is able to not only generate a holistic view of whether undercutting attacks are a threat for a wide variety of possible mempool dynamics, but it also provides guidelines on what range each of these measurable characteristics must fall within in order for the blockchain to be secure and stable long-term. Our research found that the blockchain is safe from undercutting attacks when the block size limit is small relative to the number of transactions, but the blockchain becomes more susceptible to undercutting attacks if transactions with much higher fees enter the mempool infrequently even for smaller block size limits. Moreover, we extend the logic of undercutting attacks from the original paper to show that, if the mempool dynamics are such that the undercutting occurs long-term, the tangible impact on users is that very little progress will be made as fully rational miners will end up only including one transaction per block, regardless of the total amount of available transactions.

Thesis supervisor: Neha Narula

Title: Director, Digital Currency Initiative



# Acknowledgments

Thanks to Dr. Neha Narula and Tadge Dryja for their guidance and feedback in this project. I couldn't have done it without them.



# Contents

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Title page</b>                                      | <b>1</b>  |
| <b>Abstract</b>                                        | <b>3</b>  |
| <b>Acknowledgments</b>                                 | <b>5</b>  |
| <b>List of Figures</b>                                 | <b>9</b>  |
| <b>List of Tables</b>                                  | <b>11</b> |
| <b>1 Introduction</b>                                  | <b>13</b> |
| <b>2 Background on Mining Process</b>                  | <b>21</b> |
| 2.1 Mining and Miner Revenue . . . . .                 | 21        |
| 2.2 Incentive Structure . . . . .                      | 23        |
| 2.2.1 Block Subsidy . . . . .                          | 24        |
| 2.2.2 Transaction Fees . . . . .                       | 24        |
| <b>3 Related Work</b>                                  | <b>27</b> |
| <b>4 Mining Strategies and Simulator</b>               | <b>31</b> |
| 4.1 Strategies . . . . .                               | 31        |
| 4.2 Game Theory Simulator . . . . .                    | 35        |
| 4.2.1 Original Simulator . . . . .                     | 35        |
| 4.2.2 New Augmented Simulator . . . . .                | 37        |
| <b>5 Block Size Limits</b>                             | <b>41</b> |
| 5.1 Changed Dynamic . . . . .                          | 41        |
| 5.2 Intuition Regarding Undercutting Attacks . . . . . | 42        |
| 5.3 Implementation Method . . . . .                    | 43        |
| 5.4 Normalized Block Size Limit . . . . .              | 43        |
| 5.5 Results . . . . .                                  | 43        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| <b>6</b> | <b>Fee Gradient</b>                                             | <b>47</b> |
| 6.1      | Changed Dynamic . . . . .                                       | 47        |
| 6.2      | Intuition Regarding Undercutting Attacks . . . . .              | 48        |
| 6.3      | Implementation Method . . . . .                                 | 48        |
| 6.4      | Fee Frequency . . . . .                                         | 50        |
| 6.4.1    | High Fee Frequency . . . . .                                    | 50        |
| 6.4.2    | Low Fee Frequency . . . . .                                     | 53        |
| 6.5      | Undercutting Frequency for Low Fee Frequency Dynamics . . . . . | 53        |
| 6.5.1    | Very Low NBSL . . . . .                                         | 55        |
| 6.5.2    | Low NBSL . . . . .                                              | 57        |
| 6.5.3    | High NBSL . . . . .                                             | 59        |
| 6.6      | Undercutting Profitability for Low Frequency dynamics . . . . . | 59        |
| <b>7</b> | <b>Implications if Undercutting Occurs</b>                      | <b>63</b> |
| <b>8</b> | <b>Conclusions</b>                                              | <b>65</b> |
| 8.1      | Summary . . . . .                                               | 65        |
| 8.2      | General Implications . . . . .                                  | 66        |
| 8.3      | Future Work . . . . .                                           | 67        |



# List of Figures

|      |                                                                                                           |    |
|------|-----------------------------------------------------------------------------------------------------------|----|
| 1.1  | Example of Undercutting Attack . . . . .                                                                  | 14 |
| 4.1  | Mining Strategies Relevant to Undercutting Attack Example . . . . .                                       | 32 |
| 5.1  | Plot of Normalized Block Size Limits versus Frequency of Undercutting with<br>No Fee Gradient . . . . .   | 44 |
| 6.1  | High Fee Frequency Example : A new high fee becomes available . . . . .                                   | 51 |
| 6.2  | High Fee Frequency Example: Continue Option . . . . .                                                     | 51 |
| 6.3  | High Fee Frequency Example: Undercut Option . . . . .                                                     | 51 |
| 6.4  | High Fee Frequency Example: Infeasible/Unsuccessful Undercut Attempt . .                                  | 52 |
| 6.5  | Plot of NBSL vs Undercutting Frequency for Low Fee Frequency Dynamics .                                   | 54 |
| 6.6  | Low Fee Frequency Example with Very Low NBSL : Starting state . . . . .                                   | 55 |
| 6.7  | Low Fee Frequency Example: Very Low NBSL round 1 . . . . .                                                | 56 |
| 6.8  | Low Fee Frequency Example: Very Low NBSL round 2 . . . . .                                                | 56 |
| 6.9  | Low Fee Frequency Example with Low NBSL : Starting state . . . . .                                        | 57 |
| 6.10 | Low Fee Frequency Example: Low NBSL continue option . . . . .                                             | 57 |
| 6.11 | Low Fee Frequency Example: Low NBSL Undercut Option . . . . .                                             | 58 |
| 6.12 | Fee Frequency vs Undercutting Frequency for Low NBSL . . . . .                                            | 58 |
| 6.13 | Profitability of Strategies Over Time for Low Fee Frequency and Low Fee<br>Difference Dynamics . . . . .  | 61 |
| 6.14 | Profitability of Strategies Over Time for Low Fee Frequency and High Fee<br>Difference Dynamics . . . . . | 62 |
| 8.1  | Summary of the implications on undercutting for a variety of undercutting<br>dynamics . . . . .           | 66 |



## List of Tables



# Chapter 1

## Introduction

In Bitcoin, mining is a computationally-expensive and competitive process in which miners compete to confirm transactions, build blocks of these confirmed transactions, and append them to the blockchain. These miners are then compensated for the work they have done with a combination of new coins (also referred to as the *block subsidy*) and transaction fees [2]. The long-term security and stability of Bitcoin relies on its miners behaving honestly. In order to facilitate progress in the blockchain, Bitcoin encourages miners to follow Nakamoto consensus, in which all miners agree to build blocks on the first-seen, longest chain; any long-term deviation from this consensus protocol could result in less chain progress and fewer user transactions being verified long-term.

Miners can be thought of as generally rational entities, meaning they aim to maximize their mining revenues. Thus, it is of paramount importance that the mining incentives of Bitcoin continue to align with the overall goals of the blockchain (stability and security) and that following Nakamoto consensus is still more profitable long-term for miners than deviating from the consensus protocol. Thus, any mining incentives that encourage miners to deviate from consensus constitutes a serious threat to the security of Bitcoin.

In order to understand mining incentives, it is important to understand the major factors contributing to miner revenues. Miner revenues consist of two aspects: the *block subsidy* and *transaction fees*. The block subsidy is a number of new coins that are received by the winning miner (the miner whose block made it into the final blockchain) and is received regardless of how many transactions are contained in that block; if a miner mines an empty block or a full block, they would receive the same reward in block subsidies. The reward provided by these block subsidies accounted for the vast majority of miner revenues at the beginning of the system, but this subsidy is cut in half approximately every 4 years and will eventually disappear completely in the year 2140 [2].

As the block subsidy rapidly decreases, a miner will become increasingly sensitive to the fluctuations and rewards provided by the other portion of miner revenue: transaction fees. A miner only receives the transaction fees corresponding to the transactions they are actually able to include in their block; these fees are much more variable and depend on the demand

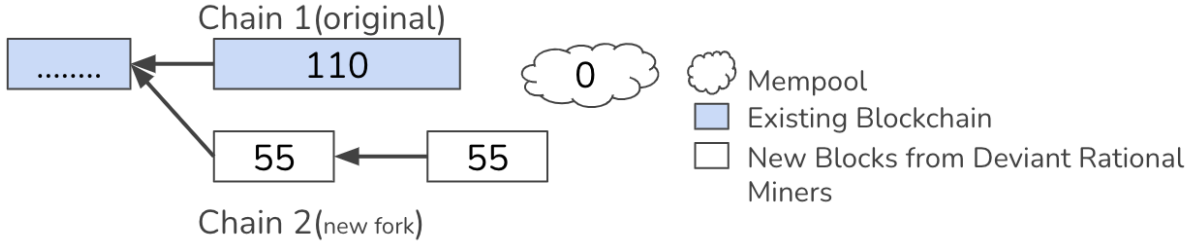


Figure 1.1: Example of Undercutting Attack

by Bitcoin users. Thus, understanding the dynamics of these transaction fees and their impact on miner behavior is becoming increasingly important; transaction fee dynamics will become a major factor in miner incentives and, therefore, the profitability or effectiveness of possible deviant mining behavior.

A previous paper written in 2016 by Carlsten et al. [1] investigated what mining behavior might occur in this highly variable transaction-fee only environment. The authors proposed the idea that once block subsidies fully disappear and transaction fees become the sole source of miner revenue, certain mining attacks may become more profitable or worthwhile. This paper hypothesized that, with a lack of sufficient block subsidies to ensure sufficient reward for all miners, the inherent variance in transaction fees would result in larger inter-block variance. In other words, one block may have a lot of transactions and would, therefore, yield a higher reward for miners, but, in the next mining round, if there were no new transactions, the next block would result in little to no reward. This inter-block reward variance could, in turn, cause rational miners to want to intentionally fork the blockchain in an attempt to steal lucrative transactions from a larger, previously-mined block; the authors called this type of attack an *undercutting attack*. These undercutting attacks would then result in miners being incentivized to repeatedly fork the blockchain, resulting in an unstable blockchain with many orphaned blocks and decreased blockchain progress. Our paper aims to examine these undercutting attacks in more detail, especially by examining how different transaction fee dynamics could affect whether the undercutting attacks would be effective. In other words, we aim to see what transaction fee dynamics would result in mining incentives that would consistently provide more reward to undercutting miners, thereby resulting in miners consistently choosing a new strategy.

Figure 1.1 shows the general intuition behind an undercutting attack that could occur as the block reward decreases rapidly. In this example, we assume that the current state of the blockchain is such that the most recently mined block has 110 transactions and 0 transactions are left to be mined in the mempool; for illustrative purposes, in Figure 1.1, we assume the transactions have the same fee and size and are, therefore, considered interchangeable.

If the next miner were to attempt to mine on this blockchain by following Nakamoto consensus, they would be unable to build a block with any reward. No transactions would be left in the mempool, meaning there would be zero transaction fees, and since we are

operating in a no-block subsidy environment, the total reward for extending the longest chain would be 0. Thus, a rational miner may realize that deviating from the consensus could yield a larger reward. Specifically, they could create chain 2 by forking the blockchain and making a new block at the previous height with only half of the available transaction fees. This new block on the forked chain would yield a reward of 55 for the miner which is more than the reward of 0 that a miner would have received had they followed consensus. In other words, forking the blockchain would be a rational action under these conditions, even though the miner is deviating from consensus. Moreover, this attack would be considered worthwhile to a miner, as the reward of 55 earned by the miner from attacking the blockchain would be significantly larger than the reward of zero that would be earned by a miner following consensus.

On this deviant chain 2, there are still 55 other transactions left in the mempool for the next miner. Thus, another rational miner would now see two chains of the same length: chain 1, which has 0 transactions available to mine, and chain 2, which has 55 transactions available to mine in the mempool. Even though chain 2 was mined by a deviant miner, this chain would provide this rational miner with more reward because chain 2 leaves a reward of 55 for the next miner whereas there is zero reward for following consensus and extending chain 1. Thus, the next rational miner would extend chain 2 as well, causing the deviant chain 2 to become the longest chain with two blocks. Thus, the deviant chain would end up being the official chain to extend, despite the fact that chain 2 is not necessarily confirming more transactions or providing more progress than chain 1, but is simply reorganizing existing already-mined transactions in a rational way.

Thus, under similar miner incentives where there aren't enough transactions to provide sufficient reward on the honest chain, rational miners would be incentivized to deviate from consensus and will instead fork the blockchain to reorganize already mined transactions that could yield themselves more reward. Repeated undercutting attacks would lead to repeated forking, as exemplified by the creation of chain 2; it would also result in orphaned blocks, such as the previously-mined block with 110 transactions that is now rendered obsolete [1]. This repeated forking would then hinder the progress and stability of the blockchain long-term, as it results in wasted mining work without measurable progress occurring on the chain.

However, the original paper by Carlsten et al. [1] that proposed these attacks assumed a specific type of transaction fee dynamic when evaluating the incentives and efficacy of these attacks. Specifically, the authors assumed a transaction fee dynamic similar to what was observed in 2016 : a low volume of transactions, blocks that weren't always full (only about 70 percent of blocks were full in 2016) [1] and transactions with equal fee rates. However, the transaction fee dynamic has changed since then due to a higher volume of transactions. For instance, due to a higher volume of transactions, it is now common for different transactions to have different fee rates and therefore yield different rewards for miners. Additionally, it is increasingly common for a high volume of transactions to lead to a backlog of unconfirmed

transactions in the mempool that must wait multiple mining rounds before being put into a block. As these transaction fee dynamics change, the corresponding miner incentives for different strategies also evolve and could affect and change the effectiveness and viability of attacks such as undercutting attacks. Indeed, previous papers such as Gong et al's paper [3] reexamined the undercutting attacks when considering dynamics where there are sufficient transactions present in the mempool such that there are enough transaction fees to fill blocks relatively consistently. They found that when there are sufficient transactions in the mempool, as defined by comparing the ratio of the previous block fee total and the next block's possible fee total to a threshold dependent on the mining power of the undercutter, the incentive to undercut is drastically reduced. Our research, on the other hand, aims to take into consideration different fee distributions and different block size limits in order to observe how the overall undercutting dynamic changes. In other words, our paper aims to generate a more holistic view of how mempool dynamics, not miner characteristics such as mining power or miner decisions, impact when undercutting attacks are a threat long-term.

Specifically, our paper provides 3 main contributions.

**Contribution 1** : *Examining how varying block size limits relative to the number of transactions entering the mempool affects the undercutting dynamic*

In 2016, the block size limit was quite large compared to the size of the mempool, as there was a lower volume of transactions; however, recently there has been a higher volume of transactions resulting in the block size limit being a smaller fraction of the mempool size. For example, in 2024, the block size limit is only 0.5 percent of the absolute size of the mempool [4]. To clarify, there is no official mempool that all miners are aware of, as different miners will have slightly different views of the mempool due to the inherent latency in the system (see Section 2.1 for more information). What we refer to as "the mempool" is actually a decently-connected node's view of the mempool; this view should have most of the general characteristics of most miners' view of the mempool, barring small deviations.

Since the block size limit is so small compared to the number of transactions available to take from the mempool, blocks are nearly always full and a large backlog of transactions form where transactions have to wait multiple rounds to be confirmed; indeed, by running our own Bitcoin node, we observed this in the actual mempool with the backlog of transactions left in the mempool averaging around 250 vMB.

This large backlog of transactions could affect the undercutting dynamic and render the undercutting attacks less effective. If there is a large amount of transactions and transaction fees available for miners to mine from in the mempool, then following Nakamoto consensus would yield sufficient reward, and there is less of an incentive to deviate from consensus and undercut.

In this paper, we aim to quantify the relationship between the block size limit and the number of transactions available to mine from by defining a related metric: the normalized block size limit (NBSL) which we define as the ratio of the raw block size limit to the number



of transactions entering the mempool each mining round. A high normalized block size limit indicates that the block size is relatively large compared to the available transactions, which in turn means that a miner can take a majority or all of the transactions from the mempool, leaving very little for the next miner; on the other hand, a low normalized block size limit indicates that there is a high volume of transactions and a miner can only mine a small fraction of the available transactions to include in their block.

In chapter 5 of this paper, we will investigate and pinpoint how varying the value of this normalized block size limit (i.e varying the presence/size of the backlog of transactions in the mempool) impacts how effective /rewarding undercutting attacks are under these varying miner incentives.

*Result: We found that for smaller NBSL values, no undercutting occurs. Specifically, if we assume an equal-fee dynamic, as long as  $NBSL \leq 1$ , no undercutting occurs, as a sufficient backlog is assembled such that there will always be enough transactions in the mempool for a miner to receive enough reward by following consensus.*

**Contribution 2** : *Examining how different fee gradients further change the undercutting dynamic.*

Now that we are considering dynamics in which block size limits are small, the idea of a fee gradient becomes increasingly relevant. When block size limits are small, transactions may have to wait multiple rounds to be mined as it may no longer be feasible for a single block to include all unconfirmed transactions in this mining round. This backlog of transactions could lead to longer confirmation time for some users, who, in turn, may now be willing to pay higher fees in order to incentivize miners to confirm their transactions earlier. Indeed we see that in 2024, there is a wide range of transaction fees, with some users willing to pay more than  $250 \text{ sat}/vB$  while others still only pay a fee of  $1 \text{ sat}/vB$  [4].

Thus, a fee gradient develops in which different transactions have different fees. The presence of a fee gradient impacts miner incentives and behavior; since different transactions have different fees, miners may have different incentives to include one specific transaction over another. For instance, miners may be incentivized to undercut a previous block to steal a specific high-fee transaction even if there is a large backlog of lower-fee transactions, as a high-fee transaction may be much more lucrative. In other words, transactions can no longer be considered interchangeable and could affect the undercutting dynamic in different ways.

We examine different types of fee gradients by splitting the idea of a fee gradient into two major factors that are most relevant when considering the undercutting dynamic: *fee frequency* and *fee difference*. The *fee frequency* is a measure of how often high fee transactions enter the mempool with a high fee frequency indicating that transactions with higher rewards are readily available and enter the mempool frequently whereas a low fee frequency indicates that these transactions with larger rewards are rare. The fee frequency could affect the range of NBSLs where undercutting attacks could occur, as rare high fees would mean that

these specific transactions could be more lucrative and could therefore encourage miners to undercut in order to compete for these scarce and lucrative transactions. The fee difference is a measure of the difference in fees between the highest and lowest fee transactions and captures how much more profitable these high fee transactions are; a high fee difference indicates that a specific transaction could be significantly more lucrative and, therefore, could result in undercutting being more successful/profitable if the deviant miner is able to acquire these extremely lucrative transactions. The fee difference primarily affects the *profitability* of undercutting, not necessarily the *frequency* of undercutting (i.e. whether or not a miner chooses to undercut short-term).

By combining and varying these two factors (fee frequency and fee difference), we can investigate a sufficiently wide variety of fee gradient dynamics and pinpoint which types of fee gradients would lead to undercutting attacks occurring and/or being profitable. We can also examine whether these fee gradients would result in undercutting attacks being more or less profitable than expected in Contribution 1. We will examine the impact of the fee gradient in Chapter 6.

*Result 1: If high fees enter the mempool infrequently, then undercutting occurs even for some smaller NBSL where undercutting didn't occur in the equal-fee dynamic.*

*Result 2: If there is a small fee difference (i.e. high fee transactions have comparable fees to other transactions), then even if undercutting occurs for smaller block size limits, undercutting attacks won't be profitable long-term.*

*Result 3: If there is a large NBSL, then undercutting attacks are profitable and a long-term threat regardless of the specific fee gradient.*

*Result 4: If there is an extremely small NBSL, then undercutting attacks will not occur even for low fee frequency, high fee difference dynamic, as the block size is so restrictive that a sufficient backlog of lucrative high-fee transactions is accumulated.*

**Contribution 3** : *Examining the impact on users if undercutting does occur in Bitcoin.*

The original paper mainly dealt with detailing how undercutting attacks work and proving that they could occur. However, it didn't clearly explain what the actual security implications of undercutting attacks were for users of Bitcoin. When discussing security implications, the original authors merely stated that the worst-case scenario of undercutting attacks would be that "consensus will break down due to... increasingly aggressive undercutting"[1] but did not provide further information on what consensus breaking down would entail nor whether there are certain assumptions or conditions that must be satisfied in order for us to reach a worst-case scenario.

Thus, in Chapter 7 of this thesis, we will investigate what exactly occurs in the blockchain and how users would be affected if undercutting attacks were to be profitable and a long-term

threat. By extending the reasoning of undercutting attacks and running a few simulations, we found that rational miners would end up taking the absolute minimum amount possible from the mempool in each round to include in their block in an effort to avoid being undercut by future miners, which in turn results in significantly reduced, but non-zero, chain progress. We also distilled the reasoning/assumptions that would be necessary to arrive at this hypothetical worst-case scenario.

*Result: If all miners are purely rational and at some point transactions stop entering the mempool for a sufficiently long period of time, then the blockchain will end up being a long chain of blocks with very few transactions per block, as miners will preemptively include the minimum possible amount of transactions in their blocks in order to avoid being undercut.*



# Chapter 2

## Background on Mining Process

In order to better understand our study of miner behavior, it is important to understand the various relevant aspects of the Bitcoin mining process and mining incentives. In this section, we will first explain what the process of mining is, the relevant components of the mining process, as well as its importance in maintaining the security and stability of the blockchain. Then, we will explain the incentive structure regarding the components of miners' revenues (block subsidies and transaction fees) as well as their evolution, characteristics, and impact on the mining dynamic.

### 2.1 Mining and Miner Revenue

At its core, Bitcoin is a shared ledger that keeps track of all coins and transactions. Due to its decentralized nature, Bitcoin and other blockchains must find a way to achieve consensus in which everyone collectively agrees on what the correct state of the blockchain is. A core assumption of the Bitcoin protocol is that the majority of miners are honest, or at least no colluding dishonest party controls the majority of the hash rate; we consider honest miners to be those that follow the Nakamoto consensus protocol. In this work, we look in more detail about what honest mining entails and how miners adopting other mining strategies could threaten the long-term security and stability.

There are many different protocols used to achieve consensus in various blockchains, but Bitcoin uses a specific proof-of-work based protocol often termed *Nakamoto Consensus*. In a proof-of-work blockchain such as Bitcoin, miners compete to discover a block by repeatedly generating a cryptographic hash based on the transactions that are included in the block as well as the hash of the previous block; these hashes are generated until they manage to generate a hash that is below a target number. Once a miner manages to find a block (i.e. generate a hash that is lower the target), they can append that block with the appropriate transactions to the blockchain and broadcast the block to other nodes. In return for the computational work that they have done in building the block, a successful miner would receive coins according to a specific incentive structure, which is explained in more detail in

the next section.

Let's take a look at the overall process of mining. Each miner has access to its own view of the sequence of previous blocks (i.e. the blockchain) and new transactions that have not yet been included in blocks and that are waiting to be confirmed (i.e. the mempool) [5]. In the real world, due to inherent latency of a decentralized system, there are discrepancies in different miners' view of the mempool and available chains; while there are differences in the the mempool views, and possibly views of the most recent blocks, the consensus protocol ensures that the nodes all share a common prefix of blocks [6]. The majority of miners will generally agree on most of the mempool and blockchain state; for the purposes of our work, we will not take into consideration the minimal variation in mempool views.

The average node's mempool of transactions is constantly changing; the mempool grows as unconfirmed transactions come in from various users, all waiting and hoping to be confirmed in the blockchain by the miners, and the mempool shrinks as miners mine these transactions and compete to put them in their blocks. Currently, in the spring of 2024, the size of the mempool has varied from 200 vMB to 550 vMB [4]. The transactions in the mempool and its size depend on the specific usage and transactions made by various Bitcoin users; transactions are submitted from the wallets of millions of different users, so the type and number of transactions is unpredictable and can change rapidly.

The specific process of Bitcoin mining is when a miner attempts to be the first to generate a cryptographic solution that matches specific criteria. The miners construct a short block header, which includes a hash commitment to all the new transactions in this block, a random nonce which is incremented for different mining attempts, and a hash of the previous block header. Each miner is attempting to be the first to generate a block header which hashes to a number less than a target. This process of mining is known as proof-of-work, as the hash function produces a number uniformly and pseudo-randomly within its output domain, and the target can be arbitrarily small. The existence of a block header which hashes to a low value is proof that a large number of nonces were tried before finding the final header, without the need to include any of the failing nonces. In other words, finding a hash below the target hash is sufficient proof of the large amount of work expended in finding the block. The difficulty to find the appropriate hash is adjusted based on how fast blocks are found. In other words, if blocks are found very quickly, then the target hash is adjusted so as to make it harder for miners to find the next block and vice versa; in this work, we ignore difficulty adjustments for simplicity. Different miners may have different computational power, meaning certain miners may be more likely to find a block earlier due to performing more trial hashes with different nonces to find a header with a hash meeting the target. The measure of a miner's probability of finding a block is approximated by a miner's hash rate or mining power. The first miner to solve this cryptographic puzzle is rewarded appropriately and all of the other miners then work on creating the next block on top of this new block. Miners with blocks in the blockchain with the work done on it, which typically translates to the longest chain with the most blocks in it, are compensated whereas miners with blocks

on any of the other chains are not [7].

Occasionally, there is a phenomenon in the blockchain called a *fork* in which two miners find a block at the same height, resulting in two different versions of the blockchain. A fork could momentarily destabilize the blockchain, as now different blockchain nodes would have a different view on which block was first, so there would not be sufficient agreement/consensus. Moreover, anyone who has mined on the original blockchain (i.e. first-seen) may find their blocks to be orphaned (i.e. no blocks are built on top of this block), so any work a miner has done by creating these orphaned blocks is useless. Orphaned blocks result in wasted work because, as a miner builds a block, the hash power that they use to compete to build a block is dependent on the hash of the previous block; thus, if the miner switches to another fork or the previous block is invalidated, then the previous computation for building the next block would be wasted (because the hash of the previous block that the miner has been using would no longer apply), and the miner's work would no longer advance the state of the blockchain [7]. Thus, orphaned blocks can artificially magnify a miner's power, as other miners waste energy and power on mining on top of blocks that don't make it into the longest blockchain.

These forks can occur accidentally if two miners happen to mine a block at the same time [6], but, what is more dangerous, is when a miner intentionally forks the blockchain. For instance, a miner might not accept and build on top of the first-seen block, but rather the miner may attempt to get his/her own block into the blockchain at the previous height instead and become the new block on which other miners mine on top of. In order for a miner's fork to become part of the official blockchain, the miner must convince enough of the other miners to agree on the consensus that their new intentionally-forked block is the correct one. According to the Nakamoto consensus protocol, the fork that is considered legitimate is the chain that has the largest total computing effort, which tends to translate to the fork that has the most number of blocks. Any blocks that are not in this winning fork with the most blocks would essentially be nullified and would not yield any reward for its miner. Thus, a miner only gets compensated if their blocks become part of the longest chain. As we see in the related paper and our research, it is possible to leverage other miner's incentives and incentive structures to convince them to deviate from consensus protocol, join an intentional fork and artificially create a longer non-honest chain.

## 2.2 Incentive Structure

As mentioned in the previous section, miners are compensated for the computational work they performed in sequencing transactions and building the block. This miner reward is the sum of two distinct sources: the block subsidy and transaction fees. The block subsidy is the only mechanism of initial coin distribution; new coins are given to miners regardless of the number of transactions mined but is exponentially decreasing over time and will therefore become a less and less important factor of miner revenues. Transaction fees, which are the revenues associated with specific transactions, are generally lower than the block subsidy

but have been becoming a larger portion as block subsidies dwindle and fees have increased; however, their inherent variance over time could result in undercutting attacks becoming more lucrative. In this section, we will examine these two aspects of miner revenues as well as their evolution over time and their impact on miner incentives.

It should be noted that both elements of a miner's revenue from a block (block subsidy and transaction fees) are not spendable until 100 blocks after the block is mined in order to ensure that the reward is not preemptively spent in a transaction that might turn out to be orphaned (i.e. initially valid but later invalidated due to its input no longer existing on the longest chain). Specifically, if a fork occurs at the miner's original block (i.e. the miner's block that yielded the block reward is orphaned) and a miner spends its rewards in a block immediately on top of that orphaned block, then a miner would have unjustly reaped the reward for its original block despite it not actually making it into the final blockchain. Unsuccessful forks that last 100 blocks without becoming the longest chain are extremely unlikely, so this 100 block delay helps reduce the impact of any reorgs, including from undercutting attacks.

### 2.2.1 Block Subsidy

The first component of a miner's revenue is the block subsidy. This block subsidy consists of newly generated coins that miners receive as incentive to mine as well as the mechanism to initially distribute the currency. As of May 2024, this block subsidy is 3.125 Bitcoins per block.

The presence of a large fixed block subsidy also helps keep the rewards that a miner earns from block to block fairly consistent; most of a miners' revenue in the past has come from this block rewards so any inherent variance in the other aspects of a miners' revenue is drastically mitigated. For instance, as of May 2024, the block reward is around 85-90 percent of the total block reward [8], so even if there were no transactions (and therefore no transaction fees) in the block, the total miner revenue would only dip by about 10-15 percent. However, Bitcoin is structured in such a way that these block rewards are exponentially decreasing; the amount that a miner gets per block is halved every 210,000 blocks or roughly every four years with the most recent halving taking place in late April 2024. The block reward will disappear completely in 2140. Thus, with this fixed subsidy rapidly dwindling, a miners' revenue will rapidly depend more and more on the other aspect of a miner's revenue: transaction fees [9].

### 2.2.2 Transaction Fees

As previously explained, there are unconfirmed transactions waiting in the mempool to be placed into a block by a miner [4]. When attempting a transaction in Bitcoin, the user can set a fee for their transaction that indicates the amount that they will pay the miner if their transaction is included in a block. When block space is limited and it is not possible for miners to confirm every available transaction in the next block, some users may want



their transactions verified and confirmed quicker; thus, they can set a higher fee for their transaction, indicating that they are willing to pay more to get their transaction confirmed; the general intuition is that the willingness to pay higher fees will incentivize miners to confirm the user’s transaction in a block before other transactions that may have lower fees. We refer to this difference in fees (i.e. reward yielded by a specific transaction) as a fee gradient.

Each miner is limited to a block of size 1MB (or 4MB with SegWit) [10], and the miner will generate revenue from the fees of the transactions they manage to put in their block. Transaction fees are typically measured by feerate (i.e. revenue per unit of size). Certain transactions can have different sizes so a transaction feerate can be thought of as the density of revenue for a certain transactions; in our work, we will use the terms feerate and fees interchangeably, as we assume all transactions have the same size (see section 4.2.2 for an explanation). Typically, transaction fees are referred to in the units of  $\text{sat}/\text{vB}$  with  $\text{sat}$  meaning a satoshi (i.e.  $10^{-8}$  of a Bitcoin, the smallest unit of currency) per unit of block space (i.e. vB which stands for virtual byte, a common unit of measurement in block size limits). Transaction fees are also highly variable, and the availability of the transactions in the mempool is highly dependent on user behavior. For instance, in January 2024, the top transactions had a fee of around 170  $\text{sat}/\text{vB}$  and there was nearly 500 vMB worth of transactions in the mempool whereas in April 2024, the top transactions had fee of only around 25 and there was a little under 200 vMB worth of transactions [4]. This highly-variable transaction fee dynamic means that one miner could have significantly more transactions to mine from than the miner who finds the next block, resulting in larger inter-block variance in miner rewards.

In 2016, only around 70 percent of blocks were filled meaning that there weren’t always enough transactions or transaction fees for miners to take [1]. This scarcity of transaction fees, if extended to a mining environment with no block reward, could result in undercutting attacks. However, it should be noted that in 2023-2024, based on observations made by running and monitoring our own node, there have so far been more than enough transactions incoming to provide a sufficient supply of transactions that nearly every block has been filled. This paper explains this different transaction fee dynamic in more detail as well as its impact on the anticipated future mining dynamics (i.e. efficacy and profitability of undercutting attacks).



# Chapter 3

## Related Work

The most important related work is the paper by Carlsten et al. published in 2016 titled “On the Instability of Bitcoin Without the Block Reward”[1]. This paper outlined many concepts that we build on top of in this paper, such as relevant mining strategies and the no-regret learning simulator used to simulate and measure the profitability of these strategies over long-periods of time in a realistic and controllable way. This paper delineated what undercutting attacks are and showed that these attacks could be profitable long-term under the transaction fee dynamic assumed in 2016 (i.e. large block size limits relative to the number of transactions in the mempool and transactions with equal fees) [1]. We build on top of the intuitions and evaluation methods of Carlsten et al’s work in this thesis.

Another major related work to our research is a paper by Gong et al. that reexamined similar assumptions regarding undercutting attacks that our research does. The authors also identified the idea that when block size limits are small enough to restrict a miner’s rewards, the dynamic of when undercutting is profitable could differ. Specifically, they found that when there is a limited bandwidth set (i.e. when there is a small fee total in the mempool on the honest chain), rational miners will always undercut; this threshold is defined to be  $\frac{\beta_U}{1-\beta_U}$  where  $\beta_U$  is the mining power of the undercutting miners. Once there are sufficient transactions in the mempool, however, undercutting could occur under certain conditions; these conditions hinge on the ratio of fees included in the previous block to the block size limit, as encapsulated by a parameter,  $\gamma$ , as well as by the mining power of the undercutting miners,  $\beta_U$ . Thus, they similarly found that “ignoring a potentially large amount of fees unclaimable in a single block can inaccurately inflate the profitability of undercutting” [3].

However, there are multiple key differences between their work and ours. First, they frame the conditions needed to avoid undercutting as something to be examined by miners during a single round of undercutting; in other words, they look at short-term undercutting profitability from the point of view of a miner, as captured by the parameter  $\gamma$ , in their analytical analysis without sufficiently examining long-term interactions.

Instead, our work aim to examine the overall mempool conditions needed to safeguard against undercutting, not necessarily looking at individual mining rounds or step-by-step

comparisons of parent blocks versus newly mined blocks. We aim to approach these conditions from a more holistic mempool view as we believe that measuring and understanding transactions in the mempool is more realistic to implement than assuming and/or restricting a specific miner’s short-term decisions.

Secondly, the paper delineates conditions based upon the undercutting miner’s hash power,  $\beta_U$ , which may not be easily calculated in a real blockchain. Instead, we aim to delineate conditions that hinge on easily measurable mempool conditions rather than having to make inferences about hash power and miner strategy, as we believe that this will allow for more transferable findings.

Moreover, the paper proposed a mitigation technique based on these conditions that they showed in simulations would result in undercutting being no more profitable than the honest strategies. However, a key caveat of this mitigation strategy is that it is only effective “if all miners consciously apply the undercutting avoidance technique” [3]. In other words, undercutting attacks can only be avoided if the miners themselves have a stake in avoiding undercutting; the paper did not provide convincing evidence that this would always be the case. Thus, rather than relying on miners to implement a mitigation avoidance technique, we aim to delineate overall mempool conditions, and perhaps techniques such as variable BSL or fee restrictions, that could be implemented in the protocol and would ensure that rational miners would not have any incentive to deviate from protocol, without relying on every miner following the avoidance technique.

Finally, the Gong et al. paper does not explicitly break down the different aspects of the fee gradient and their impact on the undercutting dynamic. While their paper does run simulations on real blockchain data, which does inherently have some transactions with differing fees, they did not pinpoint or break down that relationship, but rather focused on the block size limit aspect of the mempool conditions [3]. Our paper aims to fill this gap and to properly consider different factors of fees that could impact the dynamic.

There have also been other papers looking into possible impacts of the block reward disappearing on the mining environment. For instance, a paper by Kraner et al. used an agent-based modeling approach and framed mining as an ultimatum game in order to examine the effects of transaction arrival rate on consensus once block rewards disappeared. They found that stable consensus depends on there being enough unconfirmed transactions present (i.e. high incoming transaction rate leads to a more stable blockchain) [11]. However, these papers have not looked at undercutting attacks specifically and did not model other aspects such as fee gradients.

In addition, there have been a few papers looking into the evolution of the transaction fee environment in the past. For instance, a paper by Möser and Böhme [12], empirically analyzed transaction fees and block sizes from 2011 to mid 2014. This paper provides insight into the state of the transaction fee environment that was observed by Carlsten et al. when writing their paper in 2016 and provide some useful insight into why they made the assumptions they did at the time. For example, Möser and Böhme’s paper showed that prior

to mid-2014, the maximum block size of any block was less than a quarter of a megabyte, meaning that Carlsten’s decision to ignore block size limits was indeed quite reasonable at the time.

In order to examine more recent mempool states, we utilized a website called “Johoe’s Bitcoin mempool statistics” which gives a graphical real-time view of the unconfirmed transactions in the mempool including the number, size and fees of the transactions extending back all the way to Jan 2017 (shortly after Carlsten’s paper was written) [4]. This website provides a more current view of the state of the mempool and transaction fee dynamics that are useful in understanding what characteristics are more important nowadays.

Finally, with the publication of Carlsten’s paper, many Bitcoin-related forums and blogs began discussing its findings and relevancy [13] [14] with many pointing out that the author’s assumption that block size limits are irrelevant was especially dubious and that there may be ways to mitigate the effect of these attacks via approaches such as fee smoothing [15].



# Chapter 4

## Mining Strategies and Simulator

There are a few elements from Carlsten’s original paper that we build on top of, specifically the mining strategies that were outlined and the mining simulator. In this section, we will explain the specific mining strategies necessary to allow undercutting attacks to occur: default honest, petty honest, and undercutting miners. Specifically, in Section 4.1, we will outline the rules regarding how a miner using each strategy would choose which block to build on top and which transactions to include in the new block being built; these strategies are taken directly from Carlsten et al’s paper [1]. Then, in section 4.2, we will explain the simulator that is used to model, compare, and evaluate the different mining strategies over a long period of time. Specifically, we will explain the setup, assumptions, structure and evaluation of the mining simulator used to simulate and compare these strategies. Most of the simulator is equivalent to the simulator from Carlsten et al’s simulator; in section 4.2.1, we will specifically outline the original structure of the simulator as well as maintained assumptions that the simulator makes. In Section 4.2.2, we will explain certain general augmentations that we made to the structure and outputs of the simulator as well as some general changed assumptions. As we explain our new work in chapters 5 and 6, we will explain in more detail what new modifications we made to the simulator in order to explore different dynamics and conduct our new work.

### 4.1 Strategies

In order to understand the mining incentives that motivate undercutting attacks, it’s important to understand all of the relevant rational mining strategies and how they interact with one another. In this section, we explain the specific rules a miner using each mining strategy would use to pick which block to build on top of and which/how many transactions to include in their block based on the state of the blockchain. We will use Figure 4.1 to illustrate which types of blocks each miner would build, with blocks built by different miners being highlighted with different colors.

There are three relevant mining strategies. The first is what most would consider a

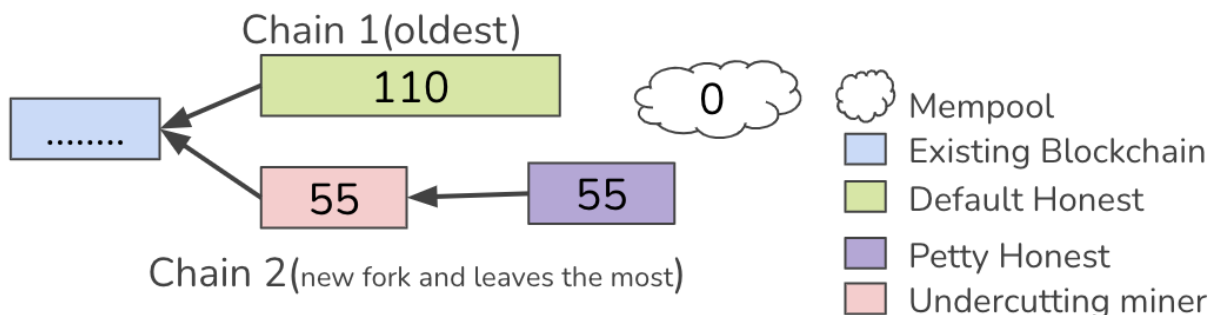


Figure 4.1: Mining Strategies Relevant to Undercutting Attack Example

normal miner; we call this type of miner a default honest miner which follows the mining strategy of the original Bitcoin software. However, when undercutting attacks occur, there are two other mining strategies that are fully rational, in that they have the potential to increase the miner’s reward, but may deviate from the default strategy. These two relevant deviant rational strategies are the undercutting and petty honest miners. The interaction between these undercutting and petty honest miners cause undercutting attacks. If both strategies are profitable long-term then undercutting may be a long-term threat to Bitcoin security; thus, it is imperative to understand exactly what the decision-making process is for these deviant miners in order to understand the proper incentive structure needed to ensure that these mining strategies are not long-term profitable.

**Default Honest** Default honest miners are assumed to be the most common type of miner in an honest blockchain. These miners follow Nakamoto consensus by always choosing to extend the longest chain (i.e. the chain with the largest number of blocks); if there are multiple forks with the same length, then the default honest miner will build a block on top of the chain with the first-seen/oldest block, thereby rewarding the miner that discovered a block first. These default honest miners are also fully greedy in that they take the highest value of transactions possible from the mempool to put in their block. For example, in Figure 4.1, the green block represents the block that the default honest would build, in that they would take all 110 transactions and build a block that extends the existing blockchain (the blue blocks). If all miners were default honest, then all miners would agree to extend the longest chain with the oldest and first-mined block; there would be no intentional forks, meaning the blockchain would be sufficiently secure.

**Petty Honest** A petty honest miner employs a somewhat similar strategy to the default honest miner, in that they also extend the longest chain as well as greedily taking as many transactions as possible from the mempool. However, when deciding which longest chain to extend, the petty honest miner extends the longest chain that **leaves the most** transactions available in the mempool, rather than the **first-seen** longest chain. In other words, if there are two possible forks with the same number of blocks, the petty honest miner would tie-



break between these two chains in a petty way by not necessarily building on the block that was built first but by building on the fork that is most profitable for themselves. If just default honest and petty honest miners are present in Bitcoin, then the blockchain is still secure as neither of these strategies actively create a fork.

However, the existence of a petty honest strategy allows non-honest deviant miners, such as undercutting miners, to succeed. This is because deviant rational miners can leverage the petty honest miners' greedy rational nature to build on top of their non honest blocks simply by leaving transactions and their fees in the mempool as incentive. In other words, because a petty honest miner doesn't necessarily build on the first-seen block (which is harder to fake when intentionally forking the blockchain), the existence of a petty honest miner allows blocks filled with already mined-transactions to become part of the longest chain. This dynamic will be exemplified in more detail in the next strategy (specifically for undercutting miners).

One should also note, that in a real blockchain, not all miners/users are guaranteed to have the same view of the blockchain and mempool, due to latency in the system, as briefly mentioned in Section 2.1. Thus, it's possible that a miner that appears to be a petty honest miner to an outside observer may actually be a default honest miner that simply saw a smaller block first, rather than the larger oldest block; similarly, it's possible that a miner that appears to be default honest is actually just a petty honest miner that didn't see the new fork in time to build on top of the more lucrative chain. This nuance is important to note because it indicates that one cannot rely on the lack of petty-honest miners to prove that undercutting does or does not occur, as it is not easy to differentiate between default and petty honest miners in a real blockchain.

**Undercutting** Undercutting miners are the second type of deviant rational miners, and they are arguably the more dangerous miners. Undercutting miners have the option to deviate from consensus by intentionally forking the blockchain and creating a block at the previous height with some already-mined transactions. Undercutting miners incentivize petty honest miners to build on their new non-honest block by leaving a portion of the transactions untouched in the mempool. Specifically, undercutting miners are parameterized by a ratio,  $r$ , which represents the fraction of the transactions that a miner will take in order to build their block; different variations of undercutting miners could have different ratios meaning that different undercutting miners may take different amounts to build their block. For example, in Figure 4.1, an undercutting miner with ratio of 0.5 would fork the blockchain and undercut the already-mined default-honest green block in the honest chain 1 and take  $0.5 \cdot mempool = 0.5 \cdot 110 = 55$  transactions to form its new block in chain 2 (the pink box in Figure 4.1), while leaving the other 55 transactions in the mempool. Then, since chain 2 has 55 transactions left in the mempool compared to the zero on the honest chain 1, the petty honest miner will build a block by extending chain 2 as represented by the purple box resulting in larger rewards for both the undercutting miner and petty honest miner but a less stable blockchain.

Specifically, the rules for undercutting miners are as follows.

1. The ratio,  $r$ , must be a fraction between 0 and 1 (exclusive). As mentioned before,  $r$  cannot be 1, as the undercutting miner has to leave some non-zero amount of transactions more in the mempool than the honest chain in order to incentivize petty honest miners to mine on top of the non-honest undercut chain
2. The undercutting miner has two options and will choose the option that gives them the most reward each time they mine based on the specific incentives/state of the blockchain at the time of mining.
  - (a) CONTINUE: If the undercutting miner chooses to continue, then they will build a block on top of the longest chain that leaves the most transaction fees in the mempool; in other words, an undercutting miner that chooses to continue essentially behaves like a petty honest miner. They will take the fraction,  $r$ , of the available transactions of the mempool on that longest chain in order to build their block. In the example in Figure 4.1, if the undercutting miner were to choose to continue, the miner would mine zero transactions and would get a reward of zero because there are zero transactions in the mempool on chain 1. If the undercutting miner chooses to continue the chain consistently, then there is no adverse effect on blockchain stability.
  - (b) UNDERCUT: If the undercutting miner chooses to undercut, then the miner will create a new fork before the most recently mined block of the longest chain and build a block at the previous height. For instance, in our example in Figure 4.1, the miner will fork the blockchain before the most recent block with 110 transactions and create fork 2 in the example. The miner will then include the fraction,  $r$ , of the mempool value of the new fork. For instance in our example in Figure 4.1, the miner has  $r = 0.5$  and the mempool value of the new fork is the sum of the value of the previous block (110) plus the value of the mempool on the longest honest chain (0); thus, the undercutting miner would take  $r \cdot mempool = 0.5 \cdot (110 + 0) = 55$  transactions and leave the rest of the transactions as incentive for petty honest miners.
    - i. In our example, the miner would choose to undercut by creating chain 2 and the pink block with 55 transactions because the undercut reward of 55 is greater than the zero reward yielded by continuing the honest chain and following consensus.
  - (c) If the reward earned by undercutting is less than or equivalent to the reward earned by continuing, then the miner will choose the continue option and will NOT fork the blockchain.

Undercutting attacks occur when this undercutting miner has sufficient incentive to deviate from consensus, and the undercut option yields more reward than continuing. These

attacks are bad for stability as it is not advancing progress in the blockchain, but rather re-mining or re-organizing already mined transactions into different blocks; for instance, the 55 transactions in the pink block are not new transactions but are part of the 110 already validated transactions in the green block on fork 1.

Note that for the scope of our research, we will only consider miners that undercut back one block; in other words, the undercutting miner will only steal fees from the previous block and not a block 2 or 3 blocks back. While it is possible for an undercutting miner to try and undercut multiple blocks back, we find sufficiently interesting results by simply examining undercutting attacks that undercut back one block. Exploring strategies that undercut more than one block back could be an interesting direction to consider for future work.

## 4.2 Game Theory Simulator

Understanding how effective different mining strategies are under a variety of different mining environments and over a long period of time is difficult to reason about without a systematic evaluation method. Thus, we build off of the game-theory based, C++ simulator from the original paper which allows us to simulate the mining process and any mining strategy over many mining rounds in a controllable environment.

The structure of how the simulator works is as follows. We will start by outlining the simulator from the original paper before explaining certain important differences.

### 4.2.1 Original Simulator

We maintain the overall structure from the simulator used in Carlsten et al’s paper [1]. In this section, we will explain the structure and mining process of the simulator, as well as certain assumptions the simulator makes that we uphold in our work.

1. At the beginning of the simulation, the total number of miners with each strategy is specified. Each miner is assigned a strategy: default honest, petty honest or a type of undercutting miner with a specific ratio,  $r$ . These strategies can be coded and controlled directly by us (see step 3). Different undercutting miners with different ratios are considered distinct strategies; for instance, an undercutting miner with a ratio of 0.2 is considered to be employing an entirely different mining strategy than undercutting miner with a ratio of 0.3; the possible undercutting ratios are defined specifically as well and are coded as separate strategy objects.
2. We define a *mining round* to be the period of time in which transactions enter the mempool, all miners attempt to mine a block, and one block is discovered, mined, and added to the blockchain. At the beginning of each mining round, we simulate a certain number of available unconfirmed transactions entering into the mempool. We can control the number of transactions directly as an input to the simulator. Note that

these are specified directly as a parameter at the beginning of the simulation. In other words, the transactions are NOT sampled from a distribution but are fixed so that the transactions entering the mempool are predictable from simulation to simulation. Also note that we simulate the blocks arriving at regular controllable intervals; in the real blockchain, blocks would arrive in a Poisson distribution but for simplicity, we assume that the time in between blocks is constant for maximum predictability/control.

3. Each miner then uses the current simulated state of the blockchain (i.e. different forks with different lengths and timestamps) and observed mempool (with various transaction and fees) to pick which block to mine on top of and which transactions to include in its own block, based on the relevant coded strategy
4. This mining process (i.e. using strategy to build a block) continues for multiple mining rounds.
5. At the end of a set number of mining rounds, which is referred to as a mining game, the total revenue each miner and mining strategy earned over the course of the game is calculated by adding the transaction fees mined by each miner for the blocks that made into the final chain (i.e. the longest chain).
6. Using the EXP3 no-regret learning algorithm (see paragraph below for a more in-depth explanation), the *strategy weight* of the strategies are updated accordingly by increasing the weight of miners with more profitable strategies and vice versa. The strategy weight measures how effective a strategy is and how profitable it is long-term.
7. Based on their strategy weights, different mining strategies are assigned to different miners for the next game (i.e. more miners are assigned to strategies with higher weights) and the process repeats from step 1 with these new strategies.
8. Steps 1-8 continue until an equilibrium of strategy weights is reached, which typically consists of hundreds of thousands of games.

This simulator uses the EXP3 no-regret learning algorithm used in the original paper [1] to simulate miners learning which strategies are most effective/profitable over the long-term. The EXP3 learning algorithm is an algorithm that assigns weights to different bandits (in this case, miners with different mining strategies) and uses the loss vector of the bandits (i.e. miners) to explore and find the configuration of experts with the least regret (i.e. most optimal distribution) [16] [17]. Over multiple games, the learning algorithm is guaranteed to reach a provable coarse-correlated equilibrium of strategies, thereby indicating which strategies would actually be profitable long-term.

A coarse correlated equilibrium is “a joint distribution over strategy profiles such that every miner gets more expected payoff by following the equilibrium than deviating to any possible strategy” [1] [16] [17]. At this equilibrium, the weight of each of these mining strategies

represent its relative effectiveness/profitability over the long run for that specific transaction fee environment and that combination of mining strategies; specifically, the weight is the probability that a rational miner would learn to adopt that strategy in order to maximize reward over the many mining games [1]. The strategies with the highest weights are the ones that will be most profitable long-term and will, therefore, be the strategies most often adopted by miners. In other words, the strategy weights measure the relative *effectiveness* or *long-term profitability* of a strategy, with a strategy weight near 1 indicating a very lucrative long-term strategy and a strategy weight near 0 indicating a very unprofitable long-term strategy.

This simulator has some assumptions regarding the mining environment that we maintain.

1. The simulator models an environment with **zero latency** so all miners have the same view of all forks and mempool at all times. Once a block is built, all miners are aware of it instantaneously. While this is not always true in the real blockchain, the majority of miners tend to have generally the same view of the chains and mempool, so we believe that this is a reasonable simplifying assumption.
2. Miners have **equivalent hash rate** (i.e. 20 miners means each miner has 5 percent of the hash rate) meaning that each discrete miner has essentially the same probability of successfully mining a block; note that there could be multiple miners with the same strategy so the aggregate hash rate of miners employing a specific strategy could vary as miners switch strategies.
3. **Transactions are considered independent.** The simulator ignores dependencies or ordering of transactions within a block. In a real blockchain, a miner has to take into consideration parent-child interactions between transactions when manufacturing a block, meaning a block may not be 100 percent full or that they may not be able to include a specific transaction because there is not enough room for a dependent transaction. This simulator ignores such interdependencies for simplicity but we believe that the overall results should still hold.

### 4.2.2 New Augmented Simulator

As part of our new work, we augmented the simulator in a couple of general ways; more detail on how these changes are implemented in the simulator can be found in later sections of the paper. In this section, we will specify which aspects of the original simulator we extended or augmented as well as certain additional or changed assumptions we made in the simulator.

1. For step 3, certain additional constraints and limitations are added to the coded strategies. Specifically, before building a block, a miner must take into consideration the block size limit; see section 5.3 for more information on how this was implemented.

2. As part of our new work, we also added functionality to mark whether the block that was built from the strategy was a result of undercutting (i.e. block was the result of the undercutter choosing the undercut option; doesn't apply if an undercutting miner chooses to continue). This marking of undercut blocks occurs in step 3.
3. In step 5 (i.e. at the end of a mining game), the simulator also calculates what fraction of the final chain consists of blocks that were a result of undercutting, based on the number of blocks in the final chain with the undercut flag. In other words, the simulator calculates how often miners are incentivized to undercut and the block that was a result of undercutting ends up in the final chain, regardless of whether or not the action of undercutting measurably increased the miner's reward relative to other strategies. This allows us to measure the *frequency* of undercutting under a set of conditions.

More formally, we define the *frequency of undercutting* for a specific mining game to be the ratio of the number of blocks in the final chain that were a result of undercutting to the total number of blocks mined over the course of the game. For example, if a miner undercuts the blockchain 10 times, all 10 blocks make it into the final chain (i.e. undercutting is successful), and there are 100 blocks mined per mining game, then the undercutting frequency would be 0.1.

Thus, using this augmented simulator, we can measure the undercutting dynamic through two aspects: frequency and efficacy of the undercutting attacks.

We define undercutting attacks as *frequent* if the calculated frequency of successful undercutting (i.e. blocks that were a result of active forking and managed to become part of the longest chain) is non-trivial for any game.

We define undercutting attacks as *profitable long-term* if the strategy weight of the undercutting strategy at the equilibrium is greater than that of the other strategies, as this indicates that all the miners agreed that undercutting is a more profitable strategy in the long run.

Note that it is possible for undercutting attacks to be *frequent* but not *profitable long-term* if there are short-term incentives to undercut but undercutting is not long-term profitable (i.e. it seems like undercutting could yield slightly more rewards but the risk to try and undercut outweighs reward). However, it is impossible for undercutting to be *profitable* if it is *not frequent*; in other words, if undercutting miners never choose to fork the blockchain then there is no need to look at the strategy weights and profitability of the undercutting strategy because if no forking occurs, undercutting miners don't actually affect the stability of the blockchain.

Moreover, our simulator adds/extends certain assumptions on top of the original simulator:

1. **All transactions have the same size.** The simulator from the original paper assumed all transactions were interchangeable (i.e. same size and same fee). While we change the assumption that all transactions have equivalent fees in Chapter 6, we maintain the assumption that all transactions have the same size. This is acceptable because we can approximate transactions with different sizes by adjusting the number and fees of the transactions.

(a) Ex: 100 transactions of size 1 is equivalent to 50 transactions with size 2

2. There are enough miners total that each miner has a sufficiently small proportion of the hash rate that **tactics like selfish mining are irrelevant**. For instance, certain strategies like selfish mining rely on miners with larger hash rates that leverage their large hash power to unduly influence the blockchain; by restricting a single miner's hash power, our simulator allows us to examine only the effects of undercutting attacks without worrying about unforeseen interactions with other types of mining attacks.

In order to accommodate different transaction fee dynamics, we mainly change the input format for the transactions (for the fee gradient) and add additional restrictions to the strategies to represent block size limits; we will explain these in more detail in the relevant sections (i.e. Sections 5.3 and 6.3. We evaluate and measure the undercutting dynamic using the frequency of undercutting and the efficacy/profitability of undercutting (strategy weights).





# Chapter 5

## Block Size Limits

The first aspect of the mining dynamic that we take into consideration that the original paper ignored was the idea of block size limits. As mentioned in section 2.2, the amount of transaction fees a miner can take in each block (and therefore the associated mining reward from transaction fees) is restricted by the size of the block; the block size limit is typically 1 MB or 4 MB with SegWit, meaning a miner can only include up to 1 MB worth of transactions in a single block. In this section, we will explain how the mempool dynamic has changed resulting in block size limits becoming a relevant factor. We will also explain how block size limits were represented in our simulator, how their relationship with the size of the mempool can be appropriately measured through a new metric called Normalized Block Size Limits (NBSL), as well as how introducing block size limits affects the undercutting dynamic.

### 5.1 Changed Dynamic

The original paper assumes that “miners always have space to include all available transactions” [1]. During times with low transaction volume such as what was occurring in 2016, the number of available transactions was sufficiently small enough that miners wouldn’t always fill their blocks. For instance, in 2016, only 70 percent of blocks were full [1]. Thus, under these conditions, it was acceptable to ignore block limits when reasoning about mining, as the scarcity of transactions tended to be the limiting factor on miner reward rather than the size of the block. However, as of spring 2024, the number of transactions in the mempool has increased significantly so that the size of the mempool has been greater than 200 vMB more consistently [4]. This new dynamic results in miners filling blocks more frequently, thus rendering block size limits more relevant to the mining dynamic as the block size limit become the primary restriction on miner reward. Thus, restricting the amount a single miner can take results in a backlog of unconfirmed transactions, as a single miner can no longer clear out all waiting transactions in the mempool. In other words, even if no new transactions enter the mempool, there are still enough transactions available in the current mempool to

allow for all miners to fill full blocks for nearly a week before the existing mempool gets depleted.

While the original paper argues that “the qualitative content of all the results continue to hold” [1], meaning that they believe that undercutting attacks will still generally work when block sizes are limited, we show that this is not entirely true. As found in the conclusions of the Gong et al. paper [3], the presence of this transaction backlog and a sufficient set of transactions from which to choose from (i.e. sufficient bandwidth set) changes the dynamic of when miners choose to undercut and could render undercutting attacks entirely ineffective for certain block size limits; we will explore this dynamic in the rest of this section by delineating what general mempool conditions (i.e. NBSL, regardless of hash power) would result in undercutting attacks being profitable long-term.

## 5.2 Intuition Regarding Undercutting Attacks

We anticipate that when a sufficient backlog is present and all transactions are of identical fee rates and size, there is no incentive to undercut as there are more than enough unconfirmed transactions waiting in the backlog for the next miner to get the same revenue by mining honestly; this is indeed supported by the Gong et al. paper [3]. Undercutting is often motivated by the scarcity of transaction fees in the mempool on the honest chain; if the remaining mempool is no longer scarce then there is no longer a need to compete for the same limited transaction fees by undercutting existing blocks as there is more than enough mining reward to be had simply by following consensus. Moreover, when there is an abundance of transactions, all miners are able to consistently hit their block limits and fully fill their blocks, leading to exactly the same revenue for the miner regardless of what strategy is taken.

In order to understand this intuition, it would be useful to examine an illustrative example. Assume there are 200 transactions in the current mempool. Without a block size limit, the first miner could take all of these 200 transactions and put it in the block. Then, when the next miner attempts to mine, there are no more transactions left for themselves, so this next miner will definitely try to undercut the first miner’s block and steal those limited transactions. If, on the other hand, the block size limit restricts the first miner to only take 100 of the 200 available transactions, there will still be a backlog of 100 transactions left in the mempool for the second miner to take from, resulting in both miners hitting their block limits and maximizing their mining reward with no incentive to deviate from consensus. Thus, under this new environment with realistic block limits, a high volume of transactions and a backlog of unconfirmed transactions waiting in the mempool, we expect there to be less incentive to undercut, and the honest strategies should be equally as effective as the undercutting strategy if block size is small enough to become a limiting factor.

## 5.3 Implementation Method

In order to implement the notion of block size limits in the simulator, the main aspect that we needed to rewrite was the mining strategies. Specifically, each mining strategy specifies the number of transactions a miner decides to put in its block. In order to take into account the limits of block sizes, we rewrote each of the strategies to cap the number of transactions the miner can include in its block to a configurable parameter that represents the block size limit; the rest of the strategy was left untouched. We can, thus, control and vary this block size limit to model different block sizes in order to see how different block size limits affect the effectiveness of the strategies. By capping the miner rewards per block to a certain limit, we rewrote the simulator so that it doesn't let strategies take more than would realistically fit in a single block.

Using these new capped strategies, we ran simulations for a range of block size limits in order to see the relationship between the block size limits and how effective undercutting strategies were relative to the baseline.

## 5.4 Normalized Block Size Limit

When taking into consideration the impact of block size limits on the undercutting dynamic, it is important to note that it is *not* the raw block size limit that is the most important but rather it is the *relationship* between the block size limit and the number of transactions entering the mempool that is important to measure and understand. For example, if there was a block size limit of 100 and 500 transactions entering the mempool each mining round, the dynamic would be pretty much identical to a block size limit of 1000 and 5000 transactions entering the mempool each round, despite the fact that the raw block size limit is 10 times greater in the latter; this is because increasing both the block size and the number of transactions simply changes the discreteness of transactions without changing the relative scarcity of miner reward from transactions.

To capture this relationship, we introduce the idea of the *normalized block size limit (NBSL)* which we define as the raw block size limit divided by the number of incoming transactions per round. This can be calculated directly based on the inputs we provide to the simulator. For instance, in the example given above, both dynamics would have an equivalent NBSL of  $100/500 = 1000/5000 = 0.2$ . Moreover, this NBSL could theoretically be calculated in the actual blockchain by directly measuring the ratio.

## 5.5 Results

We found that as long as the NBSL is less than or equal to 1, undercutting attacks are no longer effective. In other words, as long as a miner can only build a block that take less than or equal to the amount of new incoming transactions entering the mempool (i.e. amount

exiting the mempool is less than the amount entering the mempool), then the backlog of transactions will continue to grow and there will always be enough in the honest mempool to fill the next block.

We found this by running a generic simulation format with a mixture of petty honest and undercutting miners and with a set number of transactions coming in between rounds. We then varied the block size limit across a wide range of values relative to a fixed number of transactions entering the mempool. Using these parameters, we then measured which block size limit parameters specified (and which corresponding NBSL) would allow undercutting miners to have larger strategy weights compared to other honest strategies by the end of the games (i.e which NBSL values allowed undercutting attacks to be effective).

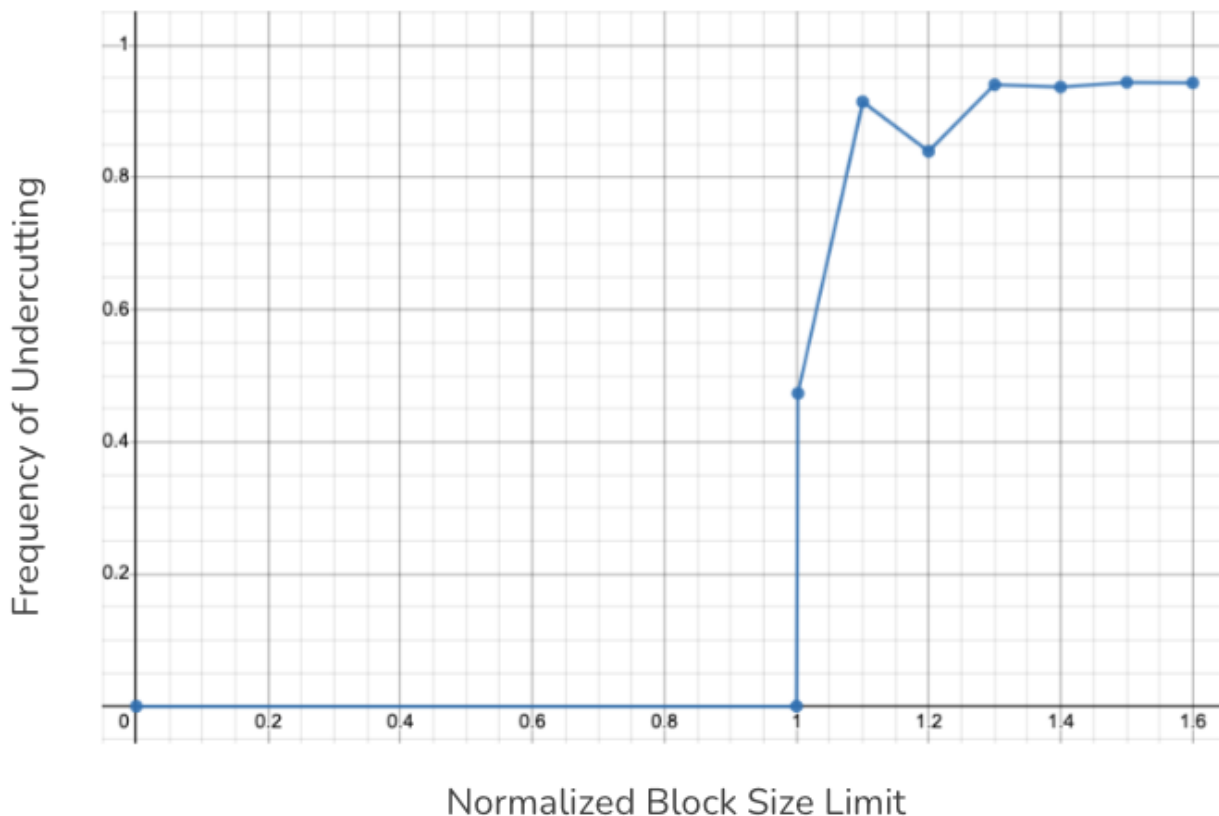


Figure 5.1: Plot of Normalized Block Size Limits versus Frequency of Undercutting with No Fee Gradient

We were able to generate Figure 5.1 by running 10 games, with each game lasting 1000 blocks; there were 20 total miners, 1 of which is a petty honest miner and the rest are undercutting miners and no switching/learning of strategies in between the games. The x-axis is the Normalized Block Size Limit and the y-axis is the frequency of undercutting as measured by the number of blocks that are a result of undercutting divided by the total number of blocks mined.

As Figure 5.1 shows, if the NBSL is less than 1, then no undercutting occurs, as the blue

line representing the frequency of undercutting is at zero. However, once the NBSL is larger than 1, then undercutting occurs frequently, as demonstrated in the original paper, as there is no longer a sufficient backlog of transactions in the mempool; indeed, in Figure 5.1, we see that the blue line representing the frequency of undercutting is near 1, thereby indicating that miners generally undercut frequently.

Thus, this figure matches the intuition we had in the beginning, as we anticipated that restricting the block size enough would lead to undercutting attacks to become ineffective.

These results are important as it shows that as long as the block size limit, which we can control in the protocol, is kept to around the appropriate ratio to the demand for transactions, we can help ensure that the mining incentives of the blockchain ensure that undercutting attacks are no longer appealing to a rational miner. In other words, by keeping the block size limit consistently below the number of transactions entering the mempool between rounds, we can hopefully discourage miners from choosing to undercut.



# Chapter 6

## Fee Gradient

The second characteristic of the mining dynamic that now becomes relevant is the idea of a fee gradient. As mentioned in section 2.2, a fee gradient can occur when there is a high volume of transactions so miners cannot mine all transactions in the immediately next block, which results in miners offering to pay different/higher transaction fees. In this section, we will explain why the fee gradient becomes relevant based on more recent dynamics. We will also explain how the idea of a fee gradient was represented and controlled in our simulator, the relevant factors that compose a specific fee gradient dynamic, as well as how different fee gradients affect the undercutting dynamic.

### 6.1 Changed Dynamic

The original paper assumes that all transactions have the same size and same transaction fee. Under the transaction fee dynamic examined in the paper, this would have indeed been true because the number of transactions was sufficiently small enough that all of the transactions available in the mempool could all be stored in a single block; thus, under this environment, users wouldn't need to pay more than the bare minimum of fees, because their transaction was essentially guaranteed to make it into the next block even without extra incentive. However, under the new high volume transaction fee environment with low NBSL values where not all transactions can make it into a block in the next round, certain users may be willing to pay higher transaction fees in order to incentivize miners to include their transactions. This leads to a transaction fee gradient that we've seen recently, with a spread of transactions from low to high fees. The presence of such a fee gradient on top of block size limits affects the mining dynamic as now two different transactions could have the same size but yield dramatically different rewards, resulting in a newfound separation in the concepts of the total number of transactions and the total value of transactions in a block.

## 6.2 Intuition Regarding Undercutting Attacks

In general, miners will want the more efficient high-fee transactions as these transactions will yield more reward for the same limited space in the block; thus, even if there are a large number of transactions in the mempool waiting to be mined, miners might still want to undercut the previous block if it contains a more lucrative transaction that provides a higher value. Thus, our intuition is that the presence of high fee transactions would increase the frequency of undercutting when the block size is limited (i.e.  $\text{NBSL} \leq 1$ ), as miners undercut in order to try and steal the high-fee transactions in the previous block, regardless of the size of the low-fee transaction backlog.

When looking at the wide range of possible fee gradient dynamics, the first aspect that we quantify is the frequency of these high fee transactions. We expect the frequency of high fee transactions to be a major factor that could impact the undercutting dynamic because the incentives to undercut a previous block hinge on the scarcity of reward in the mempool. The general intuition is that when transaction fees are scarce the threat of undercutting attacks increases; in other words, we anticipate that lowering the fee frequency will allow undercutting attacks to occur for block size limits where the blockchain had been safe from undercutting attacks in an equal fee dynamic.

The second aspect of the fee gradient dynamic that may impact the undercutting dynamic is the difference between the revenue provided by the high fee and low fee transactions. The difference in fees would help quantify how lucrative the high fee transaction is compared to the low fee transactions and whether or not stealing those transactions with higher fee rates would be more profitable long-term. Our general intuition is that a high fee difference will exacerbate the long-term threat of undercutting attacks if they occur whereas a low fee difference may discourage undercutting long-term.

If the block size is not the limiting factor (i.e.  $\text{NBSL} > 1$ ) then we expect the undercutting dynamic to be relatively unchanged in the presence of high fees, as a single high fee transaction could be replaced by the appropriate number of low fee transactions without impacting the dynamic since the size/number of transactions and therefore the specific granularity of rewards would be irrelevant; for instance, if miners could take 10 transactions, then a miner taking 5 transactions of fee rate  $1 \text{ sat}/vB$  would be equivalent to the miner taking 1 transaction of fee rate  $5 \text{ sat}/vB$ , from the point of view of the miner and mempool.

## 6.3 Implementation Method

Implementing this idea of a fee gradient in the simulator required a major overhaul. The original simulator treated the idea of the value of transactions as equivalent to the number of transactions. Specifically, groups of transactions were represented as a single integer that represented both the number and value of the transactions. However, the presence of a fee gradient requires the simulator to accurately represent the idea that some transactions



may have higher fees, meaning the value of a group of transactions could be greater than and distinct from the number of transactions in that group. Thus, these two notions (value and number) had to be separated by representing the transactions in a discrete way that highlighted the difference between them.

Ultimately, we represented transactions via a hashmap where the keys were the transaction fee and the values of the map were the number of transactions with those fees. For instance, if 4 transactions with a fee of 2 and 3 transactions of fees 1 entered the mempool in a mining round, this would be represented in our simulator as the hash-map :  $2 \rightarrow 4, 3 \rightarrow 1$ . The number of transactions could then be calculated from the values of this hash-map by summing the values of the hash-map, whereas the total revenue yielded by a group of transactions could be calculated via summing the products of the keys and values. This new discrete representation of transactions also increases the simulator's versatility by allowing the user to specify any distribution of transaction fees that they want; the user can specify exactly how many of each type of transaction enters the mining environment each round rather than being restricted to some arbitrary transaction fee distribution.

In order to accommodate this new representation of transactions, each mining strategy had to be changed to appropriately take into account two separate thresholds: *number* of transactions to fit in a block and the maximum total value of the transactions that could be taken, as calculated by taking a ratio  $r$  of total available transactions for undercutting miners. This required overwriting some of the helper functions that imposed these constraints in order to make sense of the new hash-map transaction representation.

Moreover, now that transactions are no longer interchangeable, we need an algorithm that specifies exactly how the miners choose which transactions to put in their block. Currently, what we have settled upon is that miners will greedily mine the transactions from highest fee rate to lowest fee rate; in other words, miners will take as many of the higher fee transactions as possible without violating size or value limits before moving on to the next highest fees and so on. Bitcoin Core uses a more complex algorithm that takes into consideration other factors such as transaction groups and ancestor fee rates; our algorithm does not consider a child transaction paying for a parent. For the purposes of our thesis, we will simply use this naive greedy algorithm to simulate transactions being chosen to put into a block, as we believe that, even with this straightforward selection algorithm, we can observe interesting and realistic enough results; beyond our thesis, combining more complex transaction selection algorithms with mining strategies could be an interesting avenue to further explore.

Finally, the last aspect we needed to represent was the idea that these high-fee transactions may not come in every round. The original simulator assumed that incoming transactions were constant every round; however, we needed to rewrite the function that simulated incoming transactions into the mempool in order to allow the user to control the fee frequency by controlling how often certain transactions came in. Thus, we now support the option to simulate certain transactions not entering the mempool every round, but rather once every  $n$  rounds for a specified  $n$ . We implemented this idea of periodicity by comparing

the round number to the specified periodicity in order to discern whether or not those specific transactions should come in. For instance, with a periodicity of 10, if the round number was divisible by 10, then a high fee transaction enters the mempool in the place of the usual low fee transaction. We still allowed this dynamic to be combined with the idea of specifying how many of each fee comes in. Specifically, when we are on every tenth round, we don't replace all of the incoming low fee transactions with high fee transactions, but rather, we can replace 10 percent of the total fees with high fees and leave the rest untouched. This allows for maximum flexibility and control of the exact transaction fee dynamics and allows us to model a wide variety of possible fee gradient dynamics.

## 6.4 Fee Frequency

The first relevant factor of the fee gradient that we examine is the idea of the fee frequency which is a measure of how frequently high fee transactions enter the mempool. Specifically, we define fee frequency to be the ratio of the number of rounds with a high fee transaction to the total number of mining rounds simulated. For example, if we implemented the simulator with a periodicity of 10 as explained in the previous section, then a high fee transaction only comes in once every 10 rounds(i.e. very infrequently) resulting in a low fee frequency of  $\frac{1}{10}$ .

We observed that the fee frequency impacts the ranges of NBSL for which undercutting attacks could occur. We will specifically look at two categories of fee frequencies: high and low frequencies.

### 6.4.1 High Fee Frequency

If the fee frequency is high then the transactions with higher fees come in relatively frequently; for example, a high fee frequency dynamic would entail some number of transactions with high fees entering the mempool every round.

For high fee frequency fee gradients, even if there is a high fee transaction in the previous block, it is quite likely that there will also be a similar high fee transaction entering the current mempool that could provide the same reward. Thus, a miner could get the same revenue by following consensus and mining the newly available high fee transaction from the current mempool, while leaving the high fee transaction from the previous block untouched, thus rendering undercutting unnecessary. See Figure 6.1 through 6.3 for an illustrative example of why a miner would not choose to undercut if there was a high fee frequency.

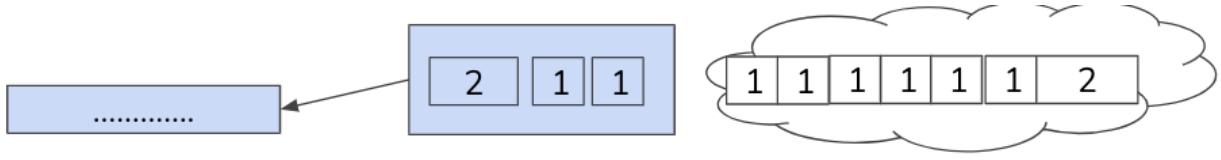


Figure 6.1: High Fee Frequency Example part 1. In this example, we assume a block size limit of 3 and we further assume that 1 high fee transaction with fee 2 enters the mempool every round. We also assume a backlog of low fee transactions (i.e. six transactions of fee rate 1) to be already present and waiting in the mempool as expected for a low BSL dynamic. Since we assume a high fee transactions enters every round, it is reasonable to assume that the previous block (represented by blue) has a high fee transaction with fee 2 as well.

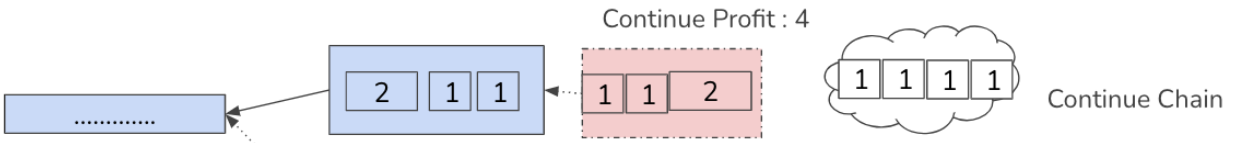


Figure 6.2: High Fee Frequency Example part 2 If miner were to choose to continue, they could build a block with the new high fee transaction of 2 and fill the rest of the block with low fee transactions, resulting in total revenue of 4; as signified by the pink box.

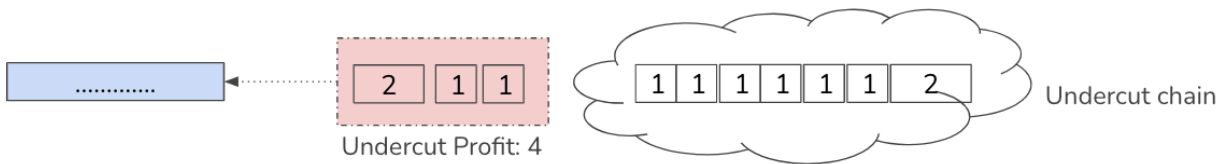


Figure 6.3: High Fee Frequency Example part 3. If a miner were to choose to undercut (pink box), they would get the same reward as the previous block, including the high-fee transaction, resulting in an equivalent revenue of 4. Since revenue is equivalent, the miner will simply choose to continue.

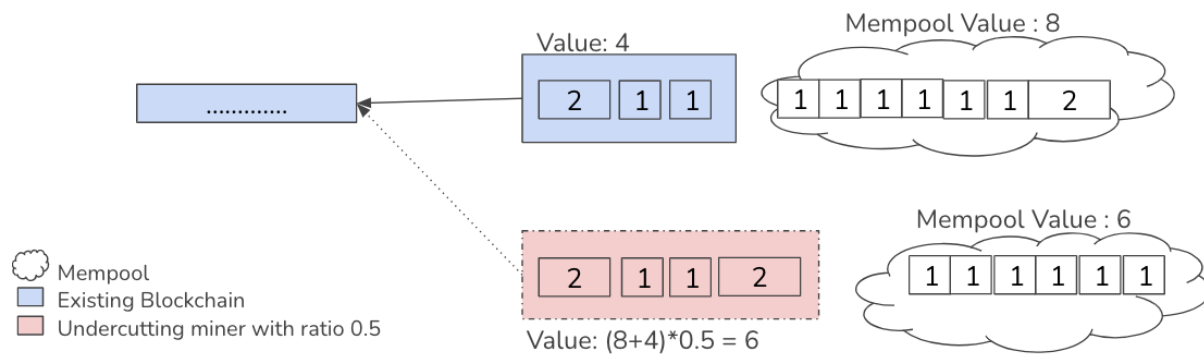


Figure 6.4: High Fee Frequency Invalid Example. Non-viable example of undercutting but including both high-fee transactions from previous block and mempool. If the undercut miner tried to include both (i.e. high fee transaction of 2 from previous block and new incoming one) then even though the new undercut block (pink) would yield higher revenue ( $6 > 8 \cdot 0.5 = 4$ ), the mempool size of the original chain would be larger ( $8 > 6$ ) meaning the petty honest would ignore the new pink undercut block and will instead build on the honest chain. In other words, even though it yields the undercutting miner more revenue, including both high-fee transactions would mean that the undercutting would never be successful.

Now, one might hypothesize that a miner may try to get a larger reward by undercutting and including the high fee transaction from the previous block and the new high fee transaction entering the current mempool. However, if a miner were to take both of these transactions, the undercutting attack would not be successful because the value of the new block with two high fee transactions would now be higher than the previous honest block that only included one high fee transaction. By definition, the undercutting miner must leave in the mempool at least the total value of the transactions that were left by the previous block in order for a miner's new undercutting block to make it to the final chain; this is because petty honest miners take the chain with the most available value in the mempool. Thus, if the undercutting miner were to take more value than that of the previous block, such as in this hypothetical case, the new undercut block would leave less transaction value in the mempool than the original block, resulting in the undercutting attack not being successful, as the petty honest miner would not build on top. This is exemplified in Figure 6.4.

Thus, the undercutting miner can't increase its revenue beyond that of the previous block, so taking both high fees (from previous block and mempool) will not succeed and thus the miner will have no incentive to undercut.

To summarize, when there is a high fee frequency, there will be enough high fee transactions available in the mempool when block size limits are small so there will be no extra incentive to undercut.

### 6.4.2 Low Fee Frequency

On the other hand, if the fee frequency is low, meaning high fee transactions are relatively scarce, this could increase the incentive to undercut. It would be possible for the previous block to have a lucrative high fee transaction but for no new high fee transactions to become available in the mempool on the honest chain, resulting in miners trying to undercut for that high fee, even if there is a large backlog of lower fee transactions.

We anticipate that this low fee frequency dynamic could cause undercutting to occur even for some smaller NBSL where undercutting might not have occurred before. Previously, for smaller NBSL values, no undercutting occurs because there is always a large backlog of transactions in the mempool, as explained in section 5. Since all transactions had the same fee previously, transactions were interchangeable, so filling a block from the backlog provided equal revenue to taking the transactions from the filled previously-mined block. However, when certain transactions have higher fees, even if the block size is sufficiently small that a large backlog of low-fee transactions accumulates in the mempool, as long as the previous block has a higher fee transactions that cannot be taken from the existing backlog of low fees, then undercutting will be incentivized, regardless of the number of low fee transactions in the mempool.

Thus, we expect the undercutting dynamic to be affected when there is a low fee frequency dynamic where high fees enter the mempool infrequently.

## 6.5 Undercutting Frequency for Low Fee Frequency Dynamics

Figure 6.5 shows the relationship between NBSL and undercutting frequency when high fees come in infrequently.

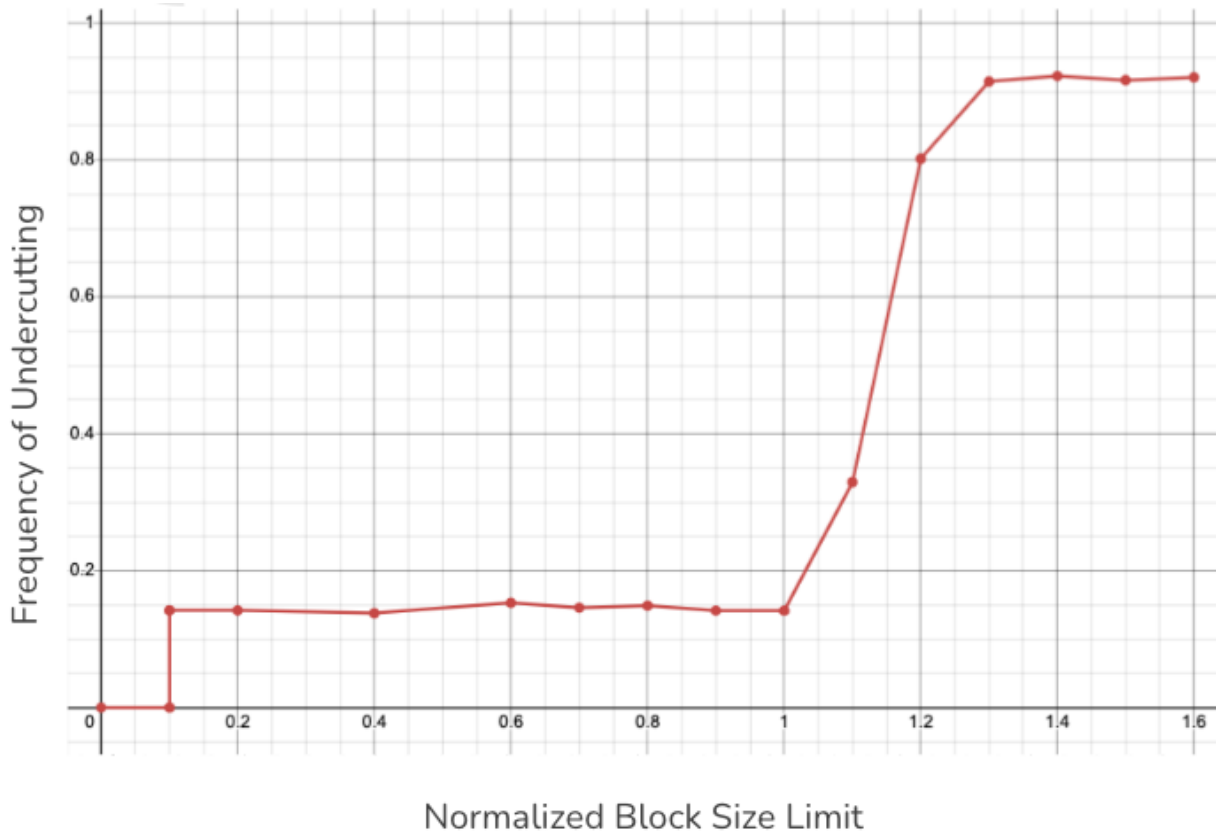


Figure 6.5: Plot of NBSL vs Undercutting Frequency for Low Fee Frequency Dynamics. For each NBSL we run 10 games with 1000 blocks per game. There were 20 total miners where 1 miner is a petty honest miner and the rest are undercutting miners. No switching/learning of strategies in between the games. 20 percent of the incoming transactions are high fee transactions with fees of 2 when they enter the mempool every other round (i.e. 50 percent of the time). The x-axis is the Normalized Block Size Limit and the y-axis is the frequency of undercutting.

In this section, we will explain the 3 NBSL ranges and their corresponding undercutting dynamics as observed in Figure 6.5: very low NBSL ( $0 - 0.1$  in this example), low NBSL ( $> 0.1$  and  $< 1$ ) and high NBSL ( $> 1$ ). We will explain what the dynamic in each of these ranges is and why, how/whether the dynamic with this fee gradient differs from the equal fee dynamic, and whether or not undercutting occurs.

### 6.5.1 Very Low NBSL

In certain cases, the NBSL is so low that the block size limit forces the high fee transactions to be spread across multiple rounds (as they can't fit in a single block), so that each round will always have enough high fee transactions in the mempool. In other words, in this case, the number of high fee transaction is so large that, even though the high fee transactions enter infrequently, there is a backlog of high fees, thereby essentially reverting to a high fee frequency dynamic. For our purposes, we define “very low” NBSL as NBSL that is less than  $\frac{x}{n}$  where a fraction  $x$  of the number of transactions are high fee transactions and they enter every  $n$  rounds.

When there is a very low NBSL, the block size must be small enough and/or the influx of high fees must be large enough that each round has exactly the same number of high fee transactions. This would then result in a backlog of high-fee transactions which lasts long enough to always provide comparable high fee transactions until the next influx of high fees, and none of the lower fee transactions would need to be mined. See Figures 6.6 through 6.8 for an illustrative example of a low fee frequency and very low NBSL dynamic.

Indeed we see that in the corresponding range in Figure 6.5 for very low NBSL, which in the case of the figure is the NBSL range between 0.1 and 1, no undercutting occurs as signified by the red line/frequency of undercutting being down at zero. Thus, for very low NBSL, no undercutting occurs, similar to the equal fee dynamic.

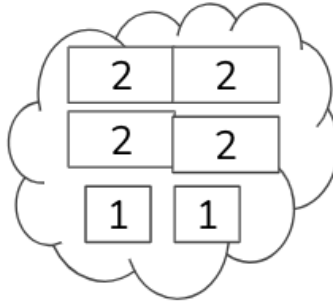


Figure 6.6: Low Fee Frequency Example with Very Low NBSL part 1. Assume 4 high fee transaction with fee 2 enters the mempool every other round (i.e.  $X = 4$ ;  $n = 2$ ) and a block size limit of  $\frac{x}{n} = 2$ .

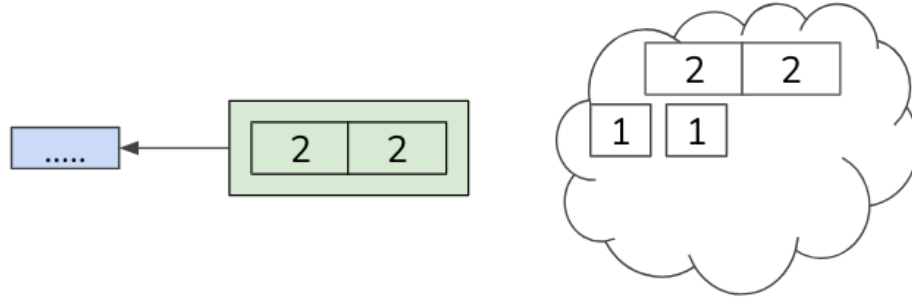


Figure 6.7: Low Fee Frequency Example with Very Low NBSL part 2. Because the miner is limited by the block size, the next default honest miner (green) will only be able to build a block with two of those high-fee transactions, leaving two more high fee transactions in the mempool for the next miner.

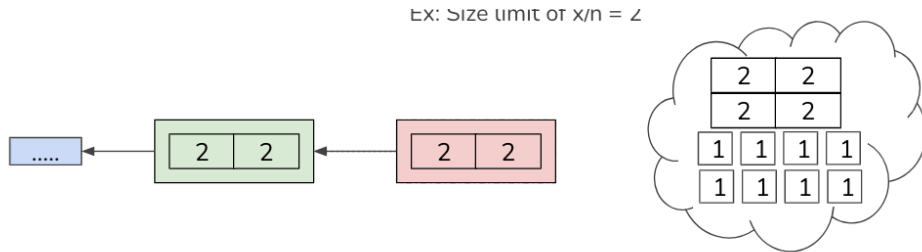


Figure 6.8: Low Fee Frequency Example with Very Low NBSL part 3. In the next mining round, the next miner (pink) would be able to take the two left-over high fee transactions, even though no new high-fee transactions came in; thus they can build an identical block with the same reward of 4. At the beginning of the next round, 4 new high fee transactions enter the mempool which is sufficient to last for two more rounds and the process repeats.



### 6.5.2 Low NBSL

The next range of NBSL that we want to look at is a low NBSL; in this dynamic there is always sufficient backlog of transactions such that all blocks are full and block space is a limiting factor BUT the NBSL is not so low that we observe the dynamic from previous section in which no undercutting because sufficient backlog of high fee transactions.

In this range, we get the most intuitive case that was explained in the low fee frequency section of section 6.4: Undercutting miners will choose to undercut in order to steal the more lucrative and efficient high fee transactions from the previous block because there are not enough high fee transactions to mine from in the honest mempool. Undercutting miners will undercut even if there is an extremely large number of low fee transactions backlogged in the honest mempool because the efficiency of the high fee transaction is more important due to limited block size; see Figures 6.9 to 6.11 for an illustrative example of such a dynamic.

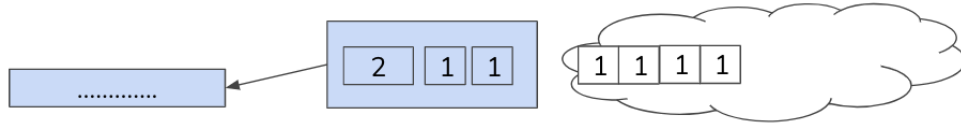


Figure 6.9: Low Fee Frequency Example with Low NBSL part 1. Assume 1 high fee transaction with fee 2 enters the mempool every other round and there is a block size limit of 3;  $NBSL = 1$ . Since only low fee transactions enter, the mempool only has a bunch of transactions with fee of 1.

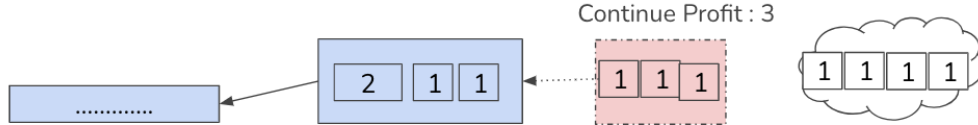


Figure 6.10: Low Fee Frequency Example with Low NBSL part 2. Since there are only low fee transactions in the mempool, if a miner were to continue the chain and build on top of the existing chain, they would only be able to take low fee transactions and get a reward of 3.

Thus, we show that undercutting *occurs* even for these low NBSL (i.e.  $NBSL \leq 1$  but  $> 0.1$ ) even though no undercutting occurred in this range for the equal fee dynamic. Indeed we see this in Figure 6.5, with the red line being near 0.15 for NBSL between 0.1 and 1; this shows that for low block size limits some undercutting will occur (i.e. undercutting occurs and the non-honest block makes it into the final chain about 15 percent of the time).

The exact frequency of successful undercutting in this case (i.e. 15 percent successful undercuts in Figure 6.5) depends on the fee frequency; the more towards the extremes the fee frequency is, the less frequently undercutting occurs as seen in Figure 6.12. This is because the incentive to undercut only occurs if the previous block has a high fee transaction

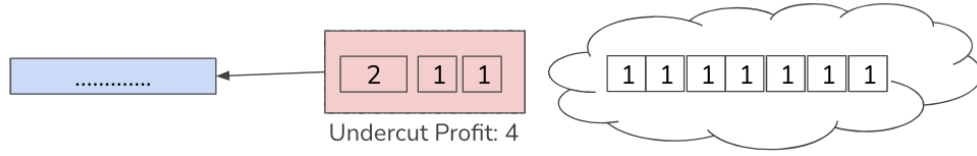


Figure 6.11: Low Fee Frequency Example with Low NBSL part 3. On the other hand, if the miner decides to undercut and take the high fee transactions from the previous block, then they could steal the efficient high fee transactions resulting in a higher undercut reward of 4. In other words, the high fee transactions in the previous block are sufficient incentive to undercut, even if the NBSL is small enough to induce a large backlog of low fees.

but the next block has less transactions. In this case, if high fees come in too frequently (near 1) then multiple rounds have high fee transactions in the mempool so the same miner reward can be taken frequently. The opposite case is true as well: if high fees come in very rarely, it's possible that all of the high fee transactions are mined in the first block after the fees come in and for every subsequent round until the next influx of high fees, neither the previous block nor the mempool have high fees, so no undercutting occurs.

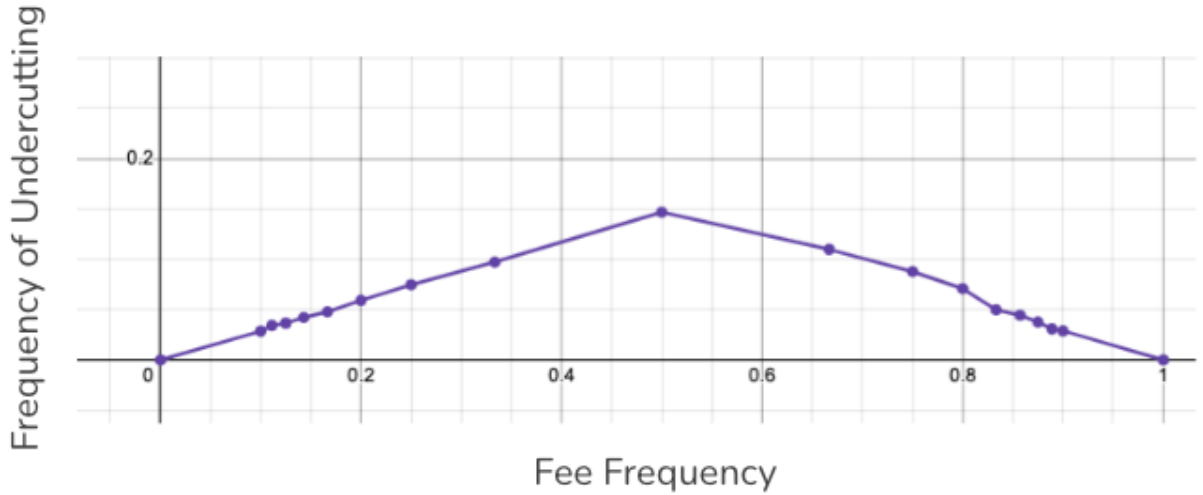


Figure 6.12: Fee Frequency vs Undercutting Frequency for Low NBSL. The purple line shows the frequency of undercutting for an NBSL of 0.3 (i.e. within the “low block size limit range”). Fee frequency, as mentioned before, is the ratio of the number of rounds with a high fee to the total number of mining rounds. This is implemented by looking at cyclicalty (i.e fee frequency of 0.25 means high fee transactions enter every 4 rounds and frequency of 0.75 means that 3 out of every 4 rounds has a high fee transaction).

### 6.5.3 High NBSL

As mentioned in section 6.2, for high NBSL, we expect that the undercutting dynamic is unchanged from the equal fee dynamic regardless of the specific fee gradient. This is because, for high NBSL, the block size is no longer a limiting factor (as not all blocks are full), so a fee gradient dynamic can be equated to a corresponding equal fee dynamic. For example, let's consider a dynamic where 5 transactions entering the mempool each round and there is a block size limit of 10 transactions (i.e.  $\text{NBSL} = 2$ ). Then, considering a fee gradient dynamic where 3 of those 5 transactions have a high fee of 2, then this dynamic is equivalent to  $3 \cdot 2 + 2 = 8$  transactions with low fees of 1 entering the mempool each round. This is because the NBSL is large enough that replacing the high fee transactions with the corresponding low-fee transactions doesn't measurably change the amount that can be taken, merely fills up the block a little bit more.

Thus, the undercutting dynamic for  $\text{NBSL} > 1$  for a low fee frequency gradient is equivalent to the undercutting dynamic for  $\text{NBSL} > 1$  with an equal fee dynamic; indeed, this is what we see in the appropriate range in Figure 6.5.

## 6.6 Undercutting Profitability for Low Frequency dynamics

Note that in the previous two sections we have been analyzing the *frequency* of undercutting which represents whether or not undercutting occurs short-term, but we have yet to analyze the *long-term profitability or efficacy* of the undercutting strategy under these dynamics.

Now, for some cases, this is already known or irrelevant. For example, for high fee frequency dynamics (section 6.4), we've shown that the undercutting dynamic is identical to the equal fee dynamic: no undercutting (and therefore not long-term profitable) for  $\text{NBSL} \leq 1$  and high frequency and efficacy of undercutting if  $\text{NBSL} > 1$ . For low fee frequency and very low block size limits, no undercutting occurs and therefore, by definition, undercutting cannot be long-term profitable. For low fee frequency and high block size limits (section 6.5.3), the dynamic is identical to the corresponding equal fee dynamic (i.e. high frequency and efficacy of undercutting just like for the equal-fee/high block size limit dynamic from the original paper).

Thus, the only dynamic that we need to non-trivially investigate the undercutting profitability is for low fee frequency and low block size limits (section 6.5.2). This is where we introduce the concept of the fee difference.

We define the fee difference as the difference between the miner rewards provided by the high fee transactions versus the rewards provided by the low fee transactions. A low fee difference would mean that the high fee transactions provide comparable miner rewards to the normal fee transactions (i.e. high fee transactions with a fee of 2) whereas a high fee difference indicates that the high fee transactions are much more lucrative and provide much

more reward than most of the other normal low-fee transactions (i.e. high fee transactions of 10). As stated before, we anticipate that the fee difference has a significant impact on the profitability of undercutting.

We've found that if there is a low fee difference, then even if high fees come in infrequently and there is a low NBSL, undercutting is not profitable long-term.

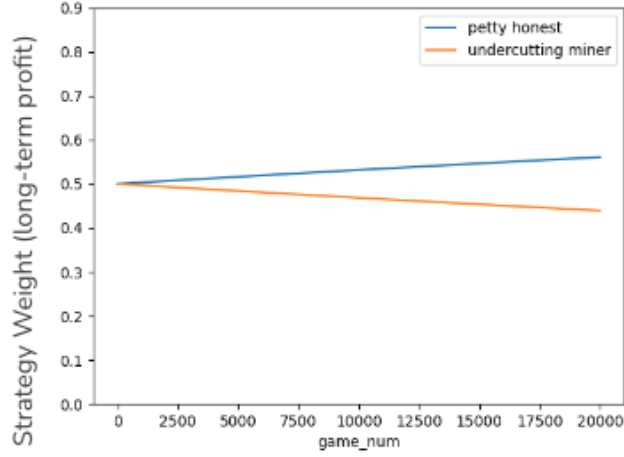


Figure 6.13: Strategy weight graph for petty honest and undercutting miners when there is a low NBSL (0.3) and low fee difference (high fee transactions have fee of 2 whereas low fee transactions have fee of 1). Strategy weights start at the same value of 0.5 meaning it's assumed that both strategies are assumed equal in the beginning. Over the course of many mining games (x-axis) and a long period of time, the strategy weights are adjusted based on miner rewards. The strategy weight (y-axis) represents the relative long-term profitability of the strategy; a weight of 0 indicates that the strategy earns very little revenue whereas a weight of 1 indicates that the strategy is much more long-term profitable than the other strategies.

Figure 6.13 shows that the long-term profitability, as represented by the strategy weight, of the petty honest strategy (blue line) is greater than that of the undercutting miner (orange line) and continues on that trend as time continues. This suggests that being honest is more long-term profitability under these conditions (low fee frequency and low fee difference). Thus, miners will realize that even though there may be incentive to undercut in the short-term, the long-term profitability is actually higher if miners simply stay honest and follow consensus. Thus, undercutting miners will eventually stop undercutting, and the blockchain will be safe long-term.

The reason that undercutting is not profitable long-term, even if it seems more profitable to the miner short-term, is that the additional miner reward gained by stealing the high fee transactions is not enough to offset the potential revenue that could have been gained from mining honestly. Let's consider an illustrative example. Let's assume that the miner reward a miner would get from undercutting, and the value of the previously mined block is 5, whereas continuing on the honest chain would only yield a reward of 4. Now, if a miner simply looks at these revenues, they will try and undercut the previous block and get a *slightly* higher revenue of 5. However, this non-honest block leaves the same amount as the previous honest block, meaning that it's not guaranteed that the new non-honest block will be mined. In the meantime, it's possible that another petty honest block will manage to mine a block on top of the honest chain, getting the slightly lower (but non-zero) reward of 4. Thus, the honest chain has now beaten the non-honest fork, as it has two blocks mined,

and the petty honest miner has been able to get the original block with reward 5 AND the reward of 4 from continuing while the undercutting miner gets zero revenue, because they wasted energy on trying to undercut to get a block with slightly higher reward and was unsuccessful, as opposed to using the opportunity to continue the chain and get a slightly lower but non-zero reward.

On the other hand, if there is a high fee difference, then the high fees are lucrative enough to make up for the lack of potential revenue; for instance, in the previous example. If the previous block had a high fee transaction of 10, then the previous block would have a reward of 13 which is much greater than the continue reward of 4; thus, the extra revenue earned from when the undercutting DOES succeed offsets the lack of opportunity.

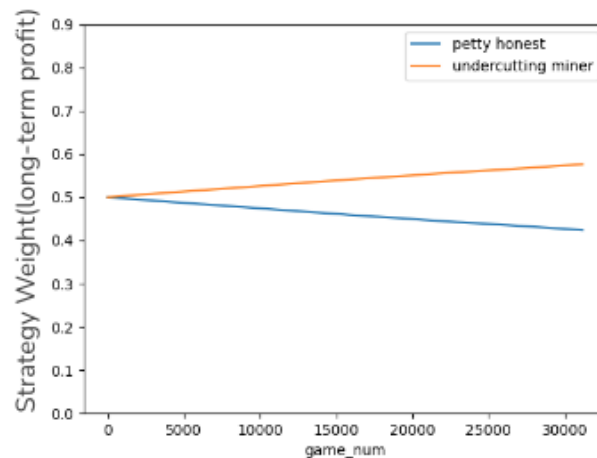


Figure 6.14: Profitability of Strategies Over Time for Low Fee Frequency and High Fee Difference Dynamics. Same axes and explanation as figure 6.13 but with a larger fee difference; high fee transactions have fee of 10 vs low fee transactions with fee of 1.

Figure 6.14 shows that the long-term profitability, as represented by the strategy weight, of the petty honest strategy (blue line) is less than that of the undercutting miner (orange line) and continues on that trend as time continues. This suggests that undercutting is indeed more long-term profitable under these conditions (low fee frequency and high fee difference). Thus, miners will continue to undercut as they see that choosing to undercut is paying off.

Thus, undercutting only is profitable long-term for lower block size limits if there is a high fee difference and high fees enter the mempool infrequently.

## Chapter 7

# Implications if Undercutting Occurs

Now that we have a more holistic view of which fee dynamics result in undercutting, the next aspect that we aim to delineate is what the tangible consequences of successful undercutting attacks are for Bitcoin users and the blockchain itself. In other words, we aim to quantify/delineate what the state of the blockchain would be.

As noted before, the original paper hypothesized that the impact on the blockchain when undercutting attacks occur would be that “consensus will break down due to... increasingly aggressive undercutting”. *Aggressive undercutting* occurs when the ratio that miners take continues to decrease so that the miners learn to take less and less of the available transactions. We will attempt to explain what “consensus breaking down” would look like in the blockchain.

For the purposes of this section, we change/loosen some previous assumptions. In this hypothetical case, we will consider the idea that a miner can undercut a block as many blocks back as desired. Moreover, we assume that at some point, zero transactions will enter the mempool for a nontrivial amount of time; in other words, there will be a nonzero number of rounds in which all miners are competing for the same limited amount of transactions that have already been mined, as there will be no new transactions entering the mempool to be mined. Thus, since a miner can undercut blocks that have been mined way in the past, any block that has been mined can be undercut, forked, and mined on top of.

We observe that when undercutting occurs, rational miners will realize that aggressive undercutting will be profitable. Let’s take a look at an example. Let’s assume that  $t$  transactions enter before users stop generating transactions for a few mining rounds, meaning that miners are now competing for the fixed  $t$  transactions. An honest miner would take the  $t$  transactions to put in its block. The next miner may realize that there are no new transactions and thus will try to undercut and take  $t - 1$  transactions, thereby leaving one more transaction in the mempool for the petty honest miner. Then, by the definition of the strategies, the next petty honest miner will build on top of the non-honest undercut block with  $t - 1$  transactions. However, what if another more aggressive undercutting miner decided to undercut the original block and take  $t - 2$  transactions. Since a block with  $t - 2$

transactions leaves even more transactions in the mempool than either the original block with  $t$  transactions or the first non-honest block with  $t - 2$  transactions, then miners will build a block on top of the block with  $t - 2$  transactions, leaving the miners of the other blocks with zero revenue. This logic can be extended; a rational miner will take the smallest possible amount to build their block in order to avoid being undercut. If a miner takes anything more than this absolute minimum (i.e. takes 2 transactions rather than 1), then some other undercutting miner will realize that they can get extra revenue by undercutting that block of 2 to get a revenue of 1 and incentivize the petty honest miner to join this new fork, leaving the original undercut miner with a revenue of zero instead of 2.

Thus, in a fully rational chain where new transactions halt for a sufficient period of time, all miners realize that they must take the absolute minimum value possible, which tends to translate to taking one transaction with the least value, in order to avoid being undercut in the future. In other words, even if there are a million transactions in the mempool, as long as its assumed that transactions will stop entering the mempool for a nontrivial amount of time at some point, a miner will still only take one transaction at this point. If a miner takes any more transactions than the absolute minimum, once those million transactions run out, some miner will fork the blockchain back at the non-optimal block's height and undercut that block and take even less transactions, resulting in zero revenue for the original miner. Since one transaction is preferable to the revenue of zero that the miner would get if they got undercut, then all miners will take the minimum amount of transactions in their block.

Thus, the implications of undercutting attacks occurring long term is that the blockchain has very slow progress, as fully rational miners will only take one transaction even if there is an extremely large backlog of transactions waiting to be mined in the blockchain. Thus, users will have extremely long wait times to get their transaction confirmed, and the blockchain will essentially end up in a near standstill. This scenario would result in very poor usability of Bitcoin, as users would struggle to transact Bitcoin and would, therefore, presumably give up on using it often; this, in turn, implies, that repeated undercutting would indeed result in a breakdown of the stability of Bitcoin as a whole. However, on the positive side, it seems reasonable to assume that incoming transactions may end up being indefinite, thereby hopefully avoiding this worst-case scenario.



# Chapter 8

## Conclusions

### 8.1 Summary

In this paper, we examined different transaction fee dynamics by modeling and evaluating the impact of two major characteristics of the mempool environment that have changed: the relationship between the block size limit and the number of transactions in the mempool, as encapsulated by the idea of the Normalized Block Size Limit, and the relevance of the fee gradient in which different transactions have different fees.

We found that if the block size limit is small relative to the number of transactions entering the mempool in each mining round, the incentive to undercut is drastically reduced. However, if a fee gradient exists and high fee transactions with significantly higher fees enter the mempool infrequently, then undercutting occurs and is profitable long-term even for smaller NBSL (i.e. when the block size limit is smaller than the number of transactions entering the mempool).

Figure 8.1 shows a tabular summary of the dynamics investigated and the implications regarding whether undercutting happens short-term and/or long-term.

In Figure 8.1, the rows represent certain ranges of NBSL and the columns represent the type of fee gradient; each cell represents the combination of these two factors that form a specific transaction fee dynamic. The green boxes indicate that for the corresponding factors (i.e. fee gradient and NBSL) the blockchain is safe from undercutting both short-term and long-term meaning no undercutting will occur. Red boxes indicate that undercutting will occur and be profitable long-term. The yellow box indicates that undercutting may be incentivized short-term but is not profitable long-term, so the blockchain is actually safe from undercutting attacks overall.

This table provides a more visual summary of the dynamics explained in sections 5 and 6. A general intuitive way to understand this table is that the top-left corner of the table is green/safe, signifying that the lower the NBSL or the higher frequency or total absence of higher fees, the more safe the blockchain is from undercutting. The lower right corners are mainly red, signifying that for larger NBSL and higher fee difference with low fee frequency,

| Normalized<br>Block Size<br>Limit | Fee Gradient    |                                 |                |                                         |                                          |
|-----------------------------------|-----------------|---------------------------------|----------------|-----------------------------------------|------------------------------------------|
|                                   |                 | None                            | High Frequency | Low Fee Frequency<br>Low Fee Difference | Low Fee Frequency<br>High Fee Difference |
|                                   | Very Low        | safe                            | safe           | safe                                    | safe                                     |
|                                   | Low( $\leq 1$ ) | safe                            | safe           | safe*                                   | Not safe                                 |
|                                   | High( $> 1$ )   | Not safe<br>(original<br>paper) | Not safe       | Not safe                                | Not safe                                 |

Figure 8.1: Summary of the implications on undercutting for a variety of undercutting dynamics

the more profitable and frequent undercutting attacks occur. The border area towards the middle of the table tends to be a bit fuzzier (i.e. yellow) where undercutting occurs short-term but does not continue happening long-term as it is not long-term profitable. Thus, this table summarizes the over-arching implications of the dynamics examined in this paper.

Moreover, our paper also explained what the impact of undercutting attacks are on users and how the blockchain may operate if undercutting attacks occur long-term. By extending the reasoning from the original paper, we explained that the blockchain will end up being long chains of blocks with the minimum possible transactions and values as miners attempt to avoid having their own blocks undercut, thereby resulting in very slow chain progress and very few of the user transactions being confirmed. In other words, if the transaction fee dynamics result in undercutting attacks being profitable and frequent long-term, the blockchain’s progress, in terms of transactions being confirmed in each block, will slow down drastically and hurt the overall stability and usability of Bitcoin.

## 8.2 General Implications

Our research advances the understanding of the future security of the Bitcoin mining environment as it gives us guidance on which factors are most relevant in keeping the blockchain safe from undercutting attacks. In other words, it gives us characteristics and ranges that certain measurable characteristics must be within in order to avoid undercutting attacks. This allows us to monitor as the mempool environment continues to evolve in order to determine when undercutting attacks are a threat and how to avoid those dynamics. For instance, in this work, we considered the NBSL, which can be easily calculated from the real blockchain at any point in time. We also considered the fee gradient, which an outside observer could somewhat approximate by observing the fees entering over multiple mining rounds. Based on this fee gradient, we could use the guidelines observed in our paper to understand what range of NBSL we need to maintain in order for the blockchain to be safe long-term (i.e. the very

low range of 0 to  $\frac{x}{n}$ ) and then dynamically change the block size limit which we can directly control in order to keep the mempool environment in the sweet spot where the incentives result in a safe mining environment where undercutting attacks won't be profitable.

Also, this thesis provides a clearer and more holistic understanding of how the blockchain will operate as the dynamic continues to evolve beyond what may be specifically expected or observed today. In other words, rather than our paper exclusively modeling and investigating the specific dynamic of 2023-2024, by looking at more generic and measurable factors, we can ensure that our overall results (i.e. low NBSL and/or high fee frequency = safe from undercutting) will still continue to be relevant even as specific values of these mempool characteristics (i.e. NBSL, fee frequency and fee difference change).

## 8.3 Future Work

While our work provides a much more holistic view of possible future undercutting dynamics, there are still many avenues that could be further explored. In this section, we will explain some possible avenues for future work as well as what we expect could occur if we were to further examine certain directions.

**Undercutting further back** We expect that looking into different dynamics when miners are able to undercut more than one block back could yield interesting results. Extending the possibility of undercutting back more than one block could increase the frequency of undercutting, as a miner would have more options to take advantage of inter-block variance further back in the blockchain. However, it is unclear if extending undercutting attacks back multiple blocks would actually increase the long-term profitability of an undercutting attack. On one hand, introducing the concept of undercutting multiple blocks back might extend the possibilities and scenarios where deviant behavior is allowed, thereby potentially increasing the frequency of undercutting. However, even if a miner is able to undercut multiple blocks back, it may be too difficult to incentivize enough petty honest miners to build on a block that far back. In other words, undercutting only works if other miners build on top of the deviant chain, but it might be difficult for the deviant chain to catch up to the height of the honest chain if undercutting multiple blocks back occurs; thus, it is possible that undercutting multiple blocks back might actually be irrelevant to the undercutting dynamic

**Latency** Currently, this paper assume zero latency in the blockchain system, such that all miners are instantaneously aware of everything that occurs in the blockchain and the mempool; this assumption simplifies understanding and modelling the system but is not entirely realistic. Adding in the idea of non-zero latency could affect the undercutting dynamic in subtle ways, as a petty honest miner may not be aware of the undercut block quickly enough and will instead only build on the honest one. Thus, our general intuition is that increasing latency may decrease the specific frequency and/or efficacy of undercutting under certain dy-

namics, though we anticipate that the general intuition of when undercutting attacks occur should still hold.

**Larger Miners** Currently, this thesis assume miners all have equivalent hash rate so all miners have an equal chance of mining. However, in reality, different miners could have different hash rates[18]. Thus, it would be an interesting avenue to explore how the undercutting dynamic would be impacted if one or two miners have larger hash rates than the other miners. We anticipate that undercutting may be less successful, as it would be more probable that a single miner could discover two blocks in a row due to having larger hash power; since, under the current assumptions, a miner would only be able to undercut one block back, a large miner discovering two blocks in a row would result in their rewards being safe from undercutting and could result in less frequent undercutting. However, if we remove the notion of a miner being restricted to only undercutting the previous block, adding in miners with slightly larger mining power may not actually affect the undercutting dynamic in a measurable way.

**Even More Transaction Fee Dynamics** While we covered two large characteristics of transactions and fee dynamics (block size limits and fee gradients), there are still many avenues left to explore. For instance, what could occur if transactions don't enter at a constant rate (i.e. number of transactions mined differs from round to round)? Moreover, we could consider seasonality in the number of transactions entering the mempool or transactions being pulled from a distribution rather than being hard-coded each time. Furthermore, it could be interesting to model real-life data to see if undercutting would be profitable in a dynamic specific to today, but without the block subsidy. All of these avenues to explore could leverage certain general ideas and evaluation methods from this thesis, and would allow for an even more thorough understanding of how the transaction fee impacts undercutting attacks and deviant mining behavior.

**Small But Nonzero Subsidy** The research so far has dealt with what occurs when the block subsidy is at zero, which will occur in 2140. However, we anticipate that some undercutting attacks could succeed, although their impact will be decreased, for some non-zero but relatively small block reward. This is due to the fact that undercutting attacks are incentivized due to inter-block variance, and it is possible that there will be sufficient variance in inter-block rewards even with a non-zero block subsidy. An interesting extension could be to run simulations to observe at what block reward (and therefore at what year) would undercutting attacks become a relevant threat. This would allow us to anticipate at what year, potentially before 2140, would we actually need to worry about undercutting attacks occurring in the real blockchain.

**Fee Smoothing** Another interesting mitigation to consider would be fee smoothing. Fee smoothing is an approach that has been hypothesized to mitigate this problem. The idea

of fee smoothing is that it is a “consensus rule that distributes mining fees between miners in a more egalitarian way, by paying each miner a function of the past block rewards” thus reducing the interblock reward variance that is inherent to incentivizing the undercutting attack [15]. It would be interesting to extend the simulator by implementing fee smoothing to investigate whether fee smoothing is actually a reasonable and successful solution to these attacks. We anticipate that fee smoothing may mitigate the frequency of these attacks, but we are unsure as to whether fee smoothing would solve the problem entirely.



# Bibliography

- [1] S. M. W. A. N. M. Carlsten H. Kalodner, “On the instability of bitcoin without the block reward,” *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016.
- [2] *Bitcoin: A peer-to-peer electronic cash system*, <https://bitcoin.org/bitcoin.pdf>, Accessed: 2024-5-10.
- [3] W. S. T. Gong M. Minaei and A. Kate, “Towards overcoming the undercutting problem,” *Financial Cryptography and Data Security*, 2022.
- [4] *Johoe’s bitcoin mempool size statistics*, [jochen-hoenicke.de/queue/#BTC](https://jochen-hoenicke.de/queue/#BTC), Accessed: 2024-5-10.
- [5] *What is the bitcoin mempool? a beginner’s explanation (2023 updated)*. [99bitcoins.com/bitcoin/mempool](https://99bitcoins.com/bitcoin/mempool), Accessed: 2024-5-10.
- [6] *Bitcoin forks explained: Hard fork vs. soft fork - phemex academy - phemex academy*. <https://phemex.com/academy/bitcoin-hard-fork-vs-soft-fork>, Accessed: 2024-5-10.
- [7] *What is bitcoin mining*, [www.investopedia.com/terms/b/bitcoin-mining.asp](https://www.investopedia.com/terms/b/bitcoin-mining.asp), Accessed: 2024-5-10.
- [8] *The mempool open source project*, <https://mempool.space/>, Accessed: 2024-5-10.
- [9] *Block reward: Definition, how they provide incentive, and future*, [www.investopedia.com/terms/b/block-reward.asp](https://www.investopedia.com/terms/b/block-reward.asp), Accessed: 2024-5-10.
- [10] *Block size / river glossary*, [river.com/learn/terms/b/block-size/](https://river.com/learn/terms/b/block-size/), Accessed: 2024-5-10.
- [11] A. S. T. C. J. T. B. Kraner S. Li, “Agent-based modelling of bitcoin consensus without block rewards,” *IEEE International Conference on Blockchain*, 2022.
- [12] M. Möser and R. Böhme, “Trends, tips, tolls: A longitudinal study of bitcoin transaction fees,” *Lecture Notes in Computer Science*, 2015.
- [13] *Bitcoin is unstable without the block reward*, [np.reddit.com/r/Bitcoin/comments/58x26m/bitcoin\\_is\\_unstable\\_without\\_the\\_block\\_reward/d93y1xh](https://np.reddit.com/r/Bitcoin/comments/58x26m/bitcoin_is_unstable_without_the_block_reward/d93y1xh), Accessed: 2024-5-10.
- [14] *Bitcoin is unstable without the block size limit - alex b. - medium*. [medium.com/@bergealex4/bitcoin-is-unstable-without-the-block-size-size-limit-70db07070a54](https://medium.com/@bergealex4/bitcoin-is-unstable-without-the-block-size-size-limit-70db07070a54), Accessed: 2024-5-10.

- [15] *On rsk's block reward smoothing*,  
<https://blog.rootstock.io/noticia/on-rsks-block-reward-smoothing/>, Accessed:  
2024-5-10.
- [16] Y. F. P. Auer N. Cesa-Bianchi and R. E. Schapire, "The nonstochastic multiarmed  
bandit problem," *SIAM Journal of Computing*, 2001.
- [17] A. Blum and Y. Mansour, "From external to internal regret," *Journal of Machine  
Learning Research*, 2007.
- [18] *Bitcoin mining pools*, <https://hashrateindex.com/hashrate/pools>, Accessed:  
2024-5-10.